

AudioPlatform

for Teensy 4.1

apApp Application Reference



Revision 1.0.1, July 31, 2025

Donald Swearingen and ProtoSupplies

Overview

AudioPlatform for Teensy 4.1 consists of two main components:

1. A complete and self-contained hardware platform for experimentation and development of simple-to-sophisticated, stand-alone, audio systems and applications for Teensy 4.1.
2. A software application, built within and upon the Arduino / TeensyDuino environment, utilizing the resources of the AudioPlatform hardware to deliver a highly-structured approach to the generation and processing of audio data.

AudioPlatform is a response to the difficulty that many potential creators of Teensy-based audio systems face: the challenge of acquiring or themselves building some sort of hardware to realize their objectives, often in the form of a “DIY” PCB board and list of components to be acquired separately by the user. Many of those with such interests simply lack the skills or resources to complete this process; and the general lack of availability of off-the-shelf hardware suitable to their projects restricts their options to the use of breadboards and/or simply-wired “perf” boards to do so.

AudioPlatform aims to address that deficit by offering a system that includes a significant set of resources for audio applications, with both hardware and software, usable out-of-the box, no soldering or wiring required.

AudioPlatform Hardware

The components of the hardware are based on both the elements of the original PJRC *Teensy Audio Shield* and additional components that fully extend the storage and IO capacity available for Teensy audio applications:

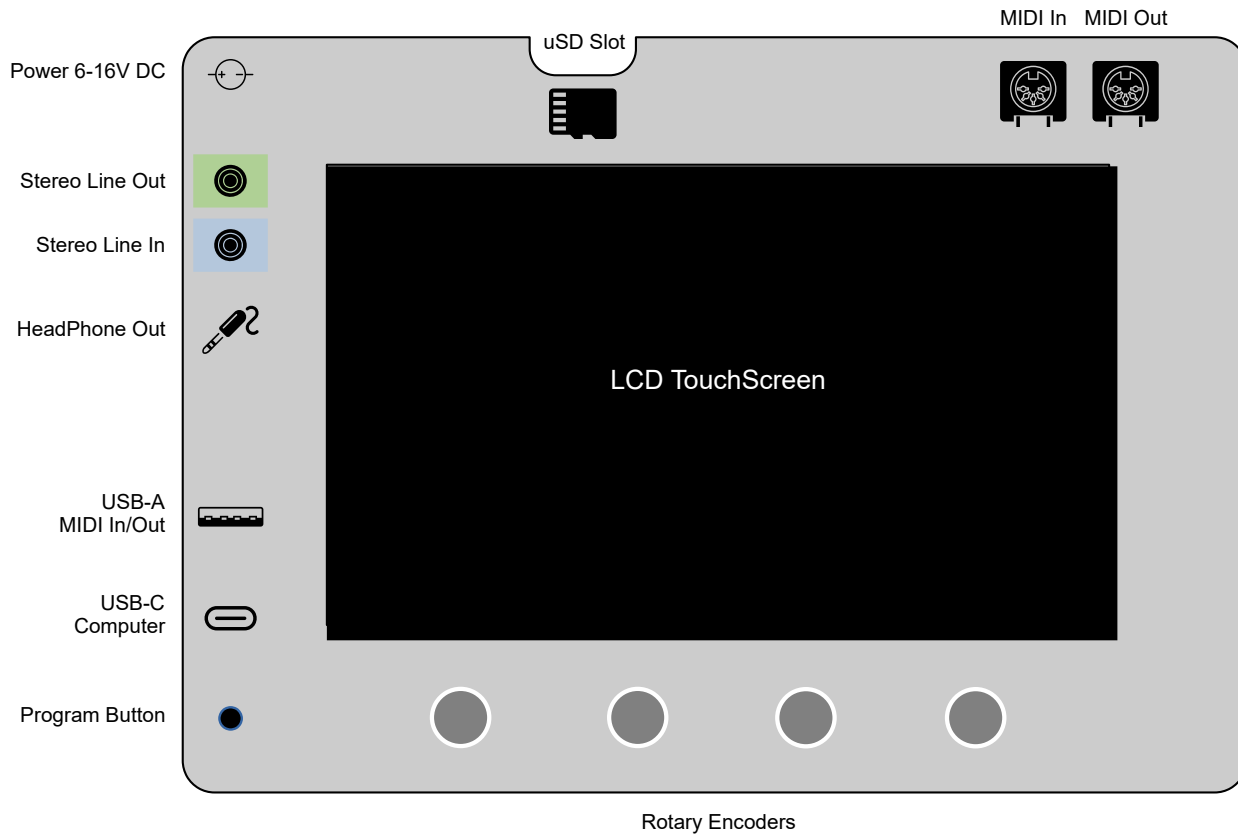
- Black Acrylic-Matte Housing and Base
- 800x480 Pixel RGB LCD Capacitive Touch Screen
- Teensy 4.1 @600 MHz with 1 MB RAM
- SGTL5000 Audio Codec
- 8 MB Non-Volatile Program Flash
- 16 MB PSRAM
- 128 MB Non-Volatile Serial Flash Storage
- Teensy 4.1 SD Card Reader, up to 32GB
- USB I/O on both MIDI Host and USB-C Connections
- Midi I/O on both USB and MIDI DIN Ports
- Audio Line In/Out on 3.5 mm Female Connectors
- Audio In/Out over USB
- Headphone Out on 3.5mm Female Connector
- 4 Rotary Encoders for Menu and Device Parameter Settings

AudioPlatform Software

The open-source AudioPlatform software application, named simply *AudioPlatform* and referred to in documentation as **apApp**, is installed and shipped with the AudioPlatform hardware. ApApp features include:

- Five Multi-Voice Sound Players: Dexed DX7 Emulator, SD WAV File Player, Serial Flash RAW File Player, Analog-Style Synthesizer, Stereo Audio Input
- 512 Dexed Voices in 16 Banks of 32 Voices, Including All Voice Banks of the Original DX7; 10 Banks Stored in Non-Volatile Program Memory; 6 User Banks Loadable from SD Card at Startup; Fully Compatible with All Banks Created in Dexed PC Application and Other Sources (1000s of Banks Available on Various WebSites)
- Stereo Reverb and Stereo Ping-Pong Delay
- All Players and Effects Simultaneously Active or Inactive Based on Preset Settings (no Mutually-Exclusive Devices)
- Player and Effects Parameters Stored in Recallable Preset Banks, 32 Presets per Bank, Unlimited Bank Storage on microSD Card with Instant Recall
- Flexible System Architecture with Individual Components (Players/Effects, Other Elements) Modifiable and/or Replaceable
- Extensive Display and System Support with C++ Libraries, Many of Which Are Application-Independent
- TouchScreen and Encoder Support for Screen Selection, Editing and Selection of Player, Effects, and SystemParameters
- Real-Time Clock Support
- Real-Time Display of Audio Levels, CPU and Memory Usage
- User Utilities for File Viewing and Management

Input/Output Connections



Connection Details

Power 6-16V DC

Power is delivered through the DC input connector from a standard 9VDC wall adapter with a center-positive (+), 2.1mm female connector. Note that power supplied through the Teensy 4.1 USB-C connection to a computer is disabled—AudioPlatform can only be powered from the DC input.

Ken: Normally ship with a 7.5V adapter

Stereo Line In/Out

Stereo Line inputs and outputs are female 3.5mm TRS connectors.

Ken: Maybe worth calling out SGTL5000 audio subsystem

HeadPhone Out

The headphone output is a 3.5mm TRS female connector.USB-A Connector

USB-A Connector

The USB-A connector is used for MIDI input from a MIDI keyboard or controller, and can supply power to a connected MIDI device.

Ken: Up to a maximum of 500mA. Higher current devices need to be externally powered.



In the first release of the AudioPlatform application (apApp), only MIDI input is supported. Support for MIDI output, and multiple input devices via a USB Hub are planned for future versions.

USB-C Connector

The USB-C connection is used exclusively for connection of AudioPlatform to a computer and supports both the Arduino IDE, and Serial IO.



Support for MIDI and Audio IO via the USB-C connection is planned for a future release of the AudioPlatform application.

Program Button

The small program button is employed in the process of downloading a pre-compiled executable (".exe") image to the AudioPlatform's Teensy 4.1, when using the Teensy Loader application (an independent component of the Arduino/Teensy environment which allows downloading independent of whether or not the Arduino IDE is active).

MicroSD Card Slot

The microSD (uSD) slot accepts a standard microSD card. Though AudioPlatform can start up and perform many of its functions without the presence of a uSD card, there are numerous features and capabilities of the apApp that depend on and require a card to be present. The types and compatibilities of various uSD media are discussed further in **SECTION REFERENCE**.

Ken: 32GB was the limitation. Now in theory can support up to 2TB but I haven't tried anything other than 32GB which is what I ship with

<https://forum.pjrc.com/index.php?threads/microsd-card-max-size-in-teensy-4-1.67242/>

Paul StofReggen 5/19/21

You need to use SdFat or the latest 1.54 betas (where SD is now just a thin wrapper for SdFat) for cards larger than 32GB.

If you use Teensyduino 1.53 or older, the old Arduino SD library supports only up to 32GB. If you're not sure which version you have, in Arduino click Help > About (or Teensyduino > About on Macintosh) to see which version is installed.

Paul StofReggen 10/17/22

Use FAT for 32GB and smaller cards, or exFat for larger than 32GB.

Theoretically up to 2TB should work, but as far as I know 1TB is the largest card anyone has tried. The largest card I own is 512GB. It works fine.

KurtE 10/17/22

Note: with SD library usage of SDFat, there are different classes you can setup to use: FatLib – which does: Fat16, Fat32 and optionally Fat12 ExFatLib – which does ExFat FsLib – which is setup to use Fat or ExFat depending on the card.

Our SD Library uses the FsLib (at least for most teensy boards)

We have format code built in, including in some of our sketches. However, you should be able to use any formatter on your PC.

I know at least at one point the developer of SDFat recommended the formatter

from: <https://www.sdcard.org/downloads/formatter/> Which should initialize and format the cards to the SD standards...

MIDI DIN In/Out

MIDI input is also possible on the 5-pin DIN connector (original MIDI standard still used with numerous current MIDI devices).



Support for MIDI output, and simultaneous use of both the MIDI DIN connection, and the USB-Host MIDI IO connection are planned for apApp.

Rotary Encoders

The 4 rotary encoders are an important and integral component of the AudioPlatform design. In the AudioPlatform application, the encoders are used to enter settings and parameters of the various devices implemented in the application. Detailed descriptions of encoder usage are outlined in **SECTION REFERENCE**.

LCD Screen

Ken: Probably worth also calling out 7" RA8875 on LCD

Documentation Ideas and Plans

Documentation Topics

- Intent/Audience
- Description
- Designer Motivations
- Deliverables
 - SD Contents
 - SD Data Maintenance Recommendations
 - Serial Flash Contents
 - Program Flash Non-Volatile System Data
- Screens and User Interface
- AudioPlatform Application Basics

AudioPlatform Documents

- Introduction
- Getting Started Guide
- User's Guide
- Application and Programmer's Reference

Introduction

Section/Topic Ideas

- What is AudioPlatform?
- Background
-

Getting Started Guide

Section/Topic Ideas

- Installation and Hookup
- Verification
- System Data SD and Flash Based
- SD and Flash Data Contents in Shipped System
- SD File Backup and Maintenance on PC (User Organization)
- Basic Menus
- Players
- Efx and MIDI Devices
- Generic MIDI Controller (as shipped) *Clarify: Compiled for generic; product does not include MIDI controller*
- Limitations (subject to future updates; ex: run-time MIDI selection)
- Instructions on how to load EXEs for other controllers and configurations
- Presets
- PresetsLive
- Screens

Ken: Should have a list of what is in the box such as:

7.5V AC/DC Adapter?

32GB MicroSD Card?

MicroSD to SD adapter?

USB-C cable?

Tilting base?

Hardcopy getting started guide?

AudioPlatform User's Manual

Section/Topic Ideas

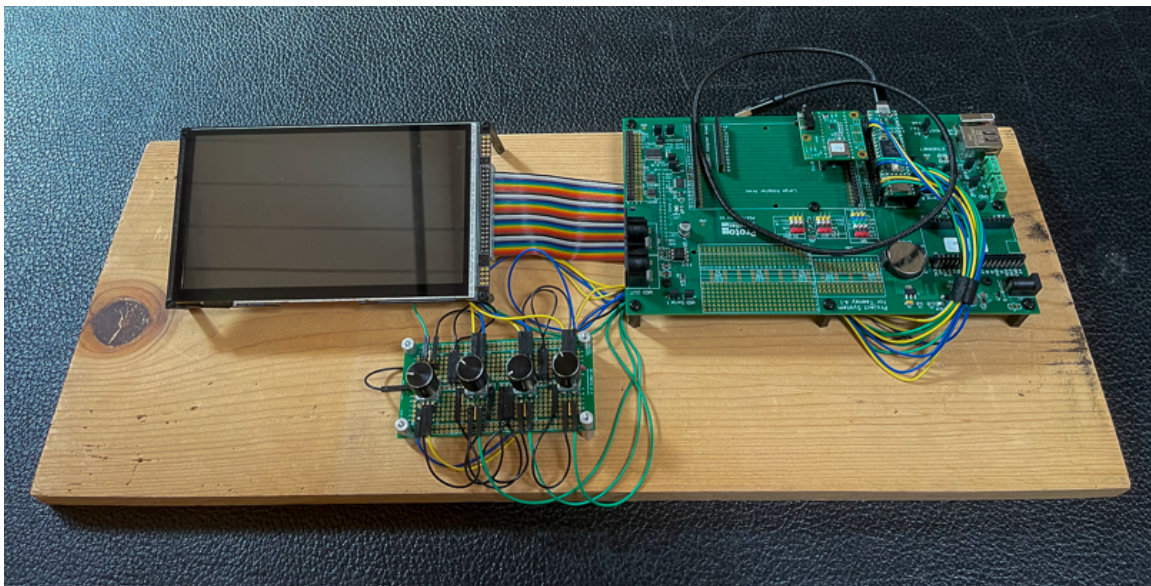
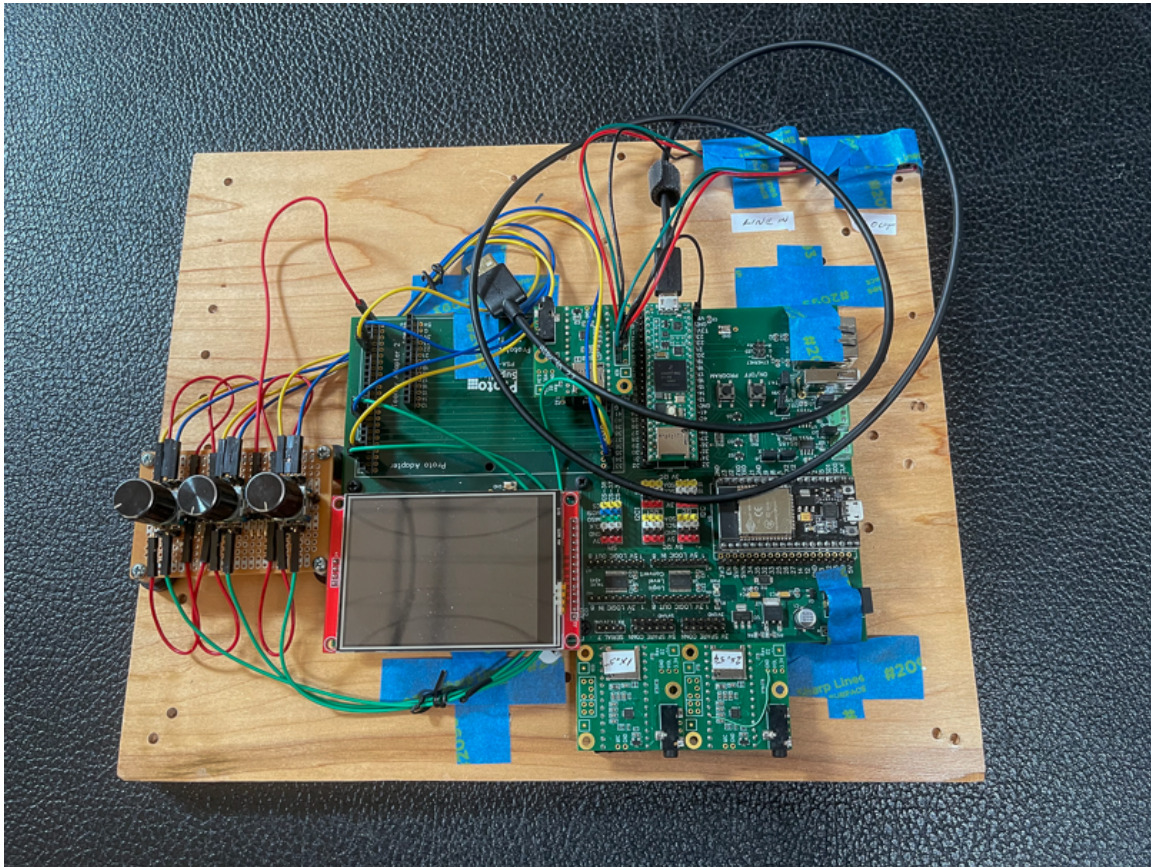
- System Overview
- Presets and Preset (Data) Manager
 - Presets Basics
 - Preset Structures (w/Diagrams)
 - Presets and Data Management
 - Data Flow
 - Preset Devices
- Basic Menus
- Player Devices
 - Player Screens
 - Player Parameters (Common and Individual)
- Audio and MIDI (Input/Controller) Devices
 - Player Screens
 - Player Parameters (Common and Individual)
- MIDI Controlllers
- Presets
- PresetsLive
- Screens

AudioPlatform Application and Programmer's Reference

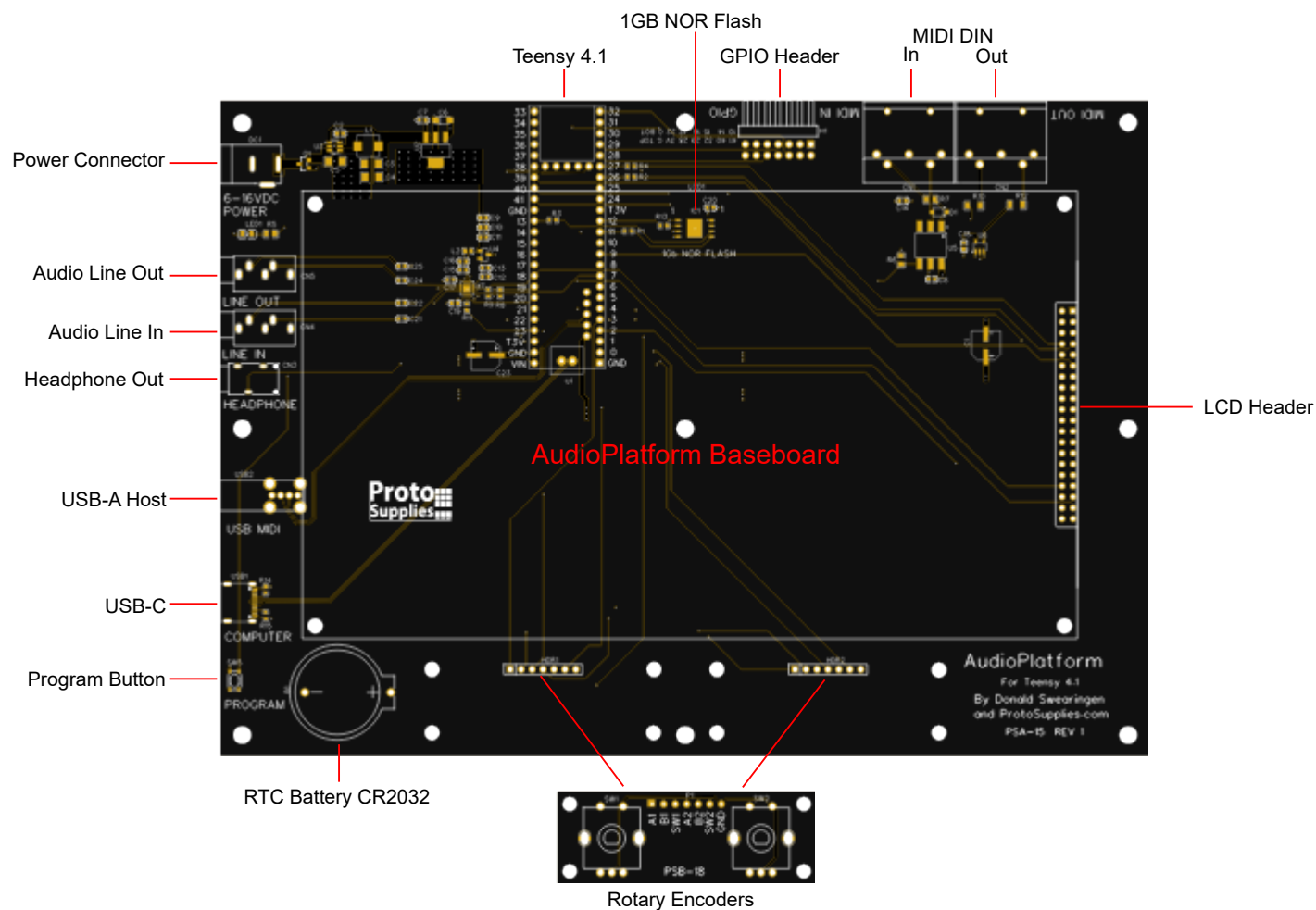
Section/Topic Ideas

- Design Principles and Goals
- Application and System Architecture (Diagrams)
- Application Elements
 - Primary Application Components
 - Application-Specific Classes (Screen and Support Classes)
 - Application-Independent Classes
 - Utility Classes
- Application Components
 - AudioPlatorm
 - Presets (To become StateMgr)
 - Screens
 - AudioDevices
 - InputDevices
- Presets/Data Management
- Arduino Mixed C/C++ Environment
 - One way function calls/callbacks C++ to C functions, but not C to C++ class method
 -

History



PCB Design



Teensy 4.1 Pin Assignments

AudioPlatform for Teensy 4.1 Pin Assignments

	Color Key	PWM	Digital Audio	Serial	Digital		Analog	I2C	SPI	CAN Bus	Power	
Baseboard						Teensy 4.1						Baseboard
Power GND					GND		VIN					VIN 5V Power
MIDI IN RX1	PWM	CRX2	CS1	RX1	0		GND					GND Ground
MIDI OUT TX1	PWM	CTX2	MISO1	TX1	1		3.3V					No Connection
RA8875 Touch INT	PWM	OUT2			2		23 A9	CRX1	MCLK1	PWM		MCLK SGTl 5000 Audio
Rotary Encoder 1 CLK_A	PWM	LRCLK2			3		22 A8	CTX1		PWM		I/O Pin 11
Rotary Encoder 1 DT_B	PWM	BCLK2			4		21 A7	RX5	BCLK1			BCLK SGTl 5000 Audio
RA8875 TFT Display CS	PWM	IN2			5		20 A6	TX5	LRCLK1			LRCLK SGTl 5000 Audio
Serial Flash CS CS	PWM	OUT1D			6		19 A5		SCL	PWM		SCL SGTl 5000 Audio / RA8875 Touch
SGTL5000 Audio DIN	PWM	OUT1A	RX2		7		18 A4		SDA	PWM		SDA SGTl 5000 Audio / RA8875 Touch
SGTL5000 Audio DOOUT	PWM	IN1	TX2		8		17 A3	TX4	SDA1			I/O Pin 9
RA8875 TFT Display RESET	PWM	OUT1C			9		16 A2	RX4	SCL1			I/O Pin 7
I/O Pin 1	PWM	MQSR	CS		10		15 A1	RX3	SPDIF IN	PWM		I/O Pin 5
Serial Flash MOSI MOSI	PWM	CTX1	MOSI		11		14 A0	TX3	SPDIF OUT	PWM		I/O Pin 3
Flash MISO MISO	PWM	MQSL	MISO		12		13 LED		SCK		PWM	SCK Serial Flash SCK
No Connection					3.3V		GND					GND Ground
Rotary Encoder 1 SWITCH	PWM	SCL2	TX6	A10	24		41 A17					I/O Pin 2
Rotary Encoder 2 CLK_A	PWM	SDA2	RX6	A11	25		40 A16					I/O Pin 4
RA8875 TFT Display MOSI1			MOSI1	A12	26		39 A15	MISO1	OUT1A			MISO1 RA8875 TFT Display
RA8875 TFT Display SCK1			SCK1	A13	27		38 A14	CS1	IN1			SWITCH Rotary Encoder 4
I/O Pin 10	PWM		RX7		28		37	CS		PWM		DT_B Rotary Encoder 4
I/O Pin 8	PWM		TX7		29		36	CS		PWM		CLK_A Rotary Encoder 4
Rotary Encoder 2 DT_B		CRX3			30		35	TX8				SWITCH Rotary Encoder 3
Rotary Encoder 2 SWITCH		CTX3			31		34	RX8				DT_B Rotary Encoder 3
I/O Pin 6		OUT1B			32		33		MCLK2	PWM		CLK_A Rotary Encoder 3

GPIO Expansion Port

				PWM	PWM			
	A17	A16	OUT1B	TX7	RX7			Teensy Pins
	41	40	32	29	28	3.3V	Gnd	
Top	2	4	6	8	10	12	14	GPIO Pins
Bottom	1	3	5	7	9	11	13	
	10	14	15	16	17	22	Gnd	Teensy Pins
	CS	A0	A1	A2	A3	A8		
	MQSR	TX3	RX3	RX4	TX4	CTX1		
	PWM	SPDIF OUT	SPDIF IN	SCL1	SDA1	PWM		
		PWM	PWM					

Color Key
Digital
PWM
Serial
Digital Audio
Analog
I2C
SPI
CAN Bus
Power

Deliverables

What's In the Box

- 7.5V AC/DC Adapter
- 32GB MicroSD Card
- MicroSD to SD adapter
- USB-C cable
- Tilting base
- Hardcopy getting started guide

microSD Card

Although the AudioPlatform application can be operated without the presence of a microSD card (generally referred to in this document as the SD Card) in the Teensy 4.1 card slot, there are a number of features of the system that are dependent on the presence of the card and its contents. These include:

- Access to Stereo WAV (".wav") audio files located on the SD card for playback by the SD Wav Player
- Access to banks of Monaural (Mono) RAW (".raw") audio files to be transferred to the internal Serial Flash storage for playback by the Flash Raw Player
- Access to user Preset Banks (".pbnk") files stored on the card
- Access to user-defined Voice Banks (".syx") for playback using the FM Synthesizer

Application Versions and Options

As discussed in detail in the *AudioPlatform User's Manual* and the *AudioPlatform Application and Programmer's Reference*, there are a number of application options that can be configured only by making modifications to the application source-code, and recompiling (rebuilding the application using the Audio IDE). However, so that users may explore and use the application without the need to set up the Arduino IDE and application software environment and carry out this process, the system is shipped with a number of pre-built "EXE" modules that can be easily downloaded into AudioPlatform without going through this process.

The 2 most important selections for getting started are 1) the selection of a generic or specific MIDI controller, and 2) whether the controller is to be connected to AudioPlatform via a USB connection, or via a standard MIDI "DIN" connection (5-pin round connector on both ends).

Default Options

- Generic MIDI Controller
- MIDI Connection via USB

Alternate Options

Alternate options can be employed both for the MIDI controller and the controller connection. Separate, installable EXE files are available for download from the link

<https://www.UrlLinkForAlternateEXEs.com>

Included are EXE modules for the following setups:

- Generic MIDI Controller, MIDI connection via USB
- Generic MIDI Controller, MIDI connection via DIN input
- Arturia Minilab3 MIDI Controller, MIDI connection via USB
- Arturia Minilab3 MIDI Controller, MIDI connection via DIN input
- Novation LaunchKey 25 MIDI Controller, MIDI connection via USB. *Note: LaunchKey 25 only includes a USB MIDI output. DIN connections are not available.*
- Novation LaunchKey 49 MK3 MIDI Controller, MIDI connection via USB
- Novation LaunchKey 49 MK3, MIDI connection via DIN input

Installing Alternate Options

Teensy Loader Application

Pre-built Teensy EXE executable files can be easily downloaded to AudioPlatform (or any compatible Teensy device connected to a computer via its standard USB connection) using the *Teensy Loader* application. Versions of Teensy Loader are available for Windows, MacOS, and Linux at the following locations:

Windows

https://www.pjrc.com/teensy/loader_win10.html

MacOS

https://www.pjrc.com/teensy/loader_mac.html

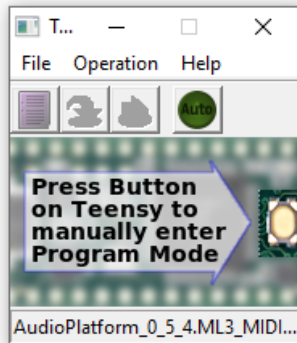
Linux (Ubuntu)

https://www.pjrc.com/teensy/loader_linux.html

Using Teensy Loader

Once you've downloaded the Teensy Loader and a suitable alternate EXE for AudioPlatform:

1. double click on the loader (Teensy.exe) to display its small screen:



2. Click on the File menu
3. Navigate to the location of the EXE to be installed and click to select (the name of the selected file will be displayed at the bottom of the loader window, as above)
4. Press the PROGRAM button at the bottom left and beneath the top panel of AudioPlatform to begin the download.

Once the download is completed, AudioPlatform will restart and display its Main Menu screen.



Among the options on the list for inclusion in a future version of the apApp are run-time selection of the MIDI controller and the connection port (USB or MIDI). In the case of the port connection, the intention is that, in the future, both ports will be active, although only a single controller may be recognized (another potential addition is the ability to use more than one controller, and perhaps a USB hub).

SD Contents

Stereo WAV Files

Bank 0

000-SoundFile.wav
001-SoundFile.wav
002-SoundFile.wav
003-SoundFile.wav
004-SoundFile.wav
005-SoundFile.wav
006-SoundFile.wav
007-SoundFile.wav
008-SoundFile.wav
009-SoundFile.wav
010-SoundFile.wav
011-SoundFile.wav
012-SoundFile.wav
013-SoundFile.wav
014-SoundFile.wav
015-SoundFile.wav
016-SoundFile.wav
017-SoundFile.wav
018-SoundFile.wav
019-SoundFile.wav
020-SoundFile.wav
021-SoundFile.wav
022-SoundFile.wav
023-SoundFile.wav
024-SoundFile.wav

Bank 1

100-SoundFile.wav
101-SoundFile.wav
102-SoundFile.wav
103-SoundFile.wav
104-SoundFile.wav
105-SoundFile.wav
106-SoundFile.wav
107-SoundFile.wav
108-SoundFile.wav
109-SoundFile.wav
110-SoundFile.wav
111-SoundFile.wav
112-SoundFile.wav
113-SoundFile.wav
114-SoundFile.wav
115-SoundFile.wav
116-SoundFile.wav
117-SoundFile.wav
118-SoundFile.wav
119-SoundFile.wav
120-SoundFile.wav
121-SoundFile.wav
122-SoundFile.wav
123-SoundFile.wav
124-SoundFile.wav

Stereo WAV Files (continued)

Bank 2

200-
201-
202-
203-
204-
205-
206-
207-
208-
209-
210-
211-
212-
213-
214-
215-
216-
217-
218-
219-
220-
221-
222-
223-
224-

Bank 3

300-
301-
302-
303-
304-
305-
306-
307-
308-
309-
310-
311-
312-
313-
314-
315-
316-
317-
318-
319-
320-
321-
322-
323-
324-

Mono RAW Files

Bank 0

000-
001-
002-
003-
004-
005-
006-
007-
008-
009-
010-
011-
012-
013-
014-
015-
016-
017-
018-
019-
020-
021-
022-
023-
024-

Bank 1

100-
101-
102-
103-
104-
105-
106-
107-
108-
109-
110-
111-
112-
113-
114-
115-
116-
117-
118-
119-
120-
121-
122-
123-
124-

Mono RAW Files (continued)

Bank 2

200-
201-
202-
203-
204-
205-
206-
207-
208-
209-
210-
211-
212-
213-
214-
215-
216-
217-
218-
219-
220-
221-
222-
223-
224-

Bank 3

300-
301-
302-
303-
304-
305-
306-
307-
308-
309-
310-
311-
312-
313-
314-
315-
316-
317-
318-
319-
320-
321-
322-
323-
324-

SD Card Usage and Management

Although the presence of an SD card is not required to use many of the basic features of apApp, the full extent of application capabilities is only possible when an SD card is present in the Teensy 4.1 microSD slot.

Because of this and various other considerations, it is considered fundamental to the full benefits of AudioPlatform that the contents of the SD card be managed in a well-organized and careful manner.

Other than the capability to name, store, and load preset banks to/from the sdCard, apApp makes no provision of sdCard file and directory management. It is thus necessary that users organize and manage soundfile resources and SD card contents on a PC (Windows, Mac, Linux) using various available applications and/or utilities. The renaming of a collection of audio files to conform to the naming requirements for SD card audio files in apApp is perhaps the primary issue in this connection,

and it is necessary to use some appropriate means of doing so. There are numerous paid and free applications that can be used for this purpose. Examples include:

PC-Based File Management Utilities

Windows

- Microsoft PowerToys PowerRename
- Advanced Renamer
- Bulk Rename Utility
- File Renamer
- Metamorphose 2

MacOS

- Finder's Built-in Feature
- Transnomino
- NameChanger
- Metamorphose 2

Linux

- GPRename
- pyRenamer
- Thunar
- Metamorphose 2

SD Card Format

SD Card Content

File Naming

In order to be recognized in apApp, WAV and RAW audio filenames must follow a specific convention that indicates which of the 4 sample banks the file is to be located in, and which MIDI note number (0-99, 100-127 ignored) will play the sample.

The convention dictates that the sample filename must begin with a 3-digit number (leading 0s required) followed by a dash '-' and the filename, and ending with the suffix .wav or .raw as the case may be. The 3 digits indicate both the bank number:

- 000-099 Bank 0, MIDI notes 0-99

- 100-199 Bank 1, MIDI notes 0-99
- 200-299 Bank 2, MIDI notes 0-99
- 300-399 Bank 2 MIDI notes 0-99

Examples

Bank 0

- 000-Note0_Sample.wav
- 060-MiddleC_Sample.wav
- 099-Note99_Sample.wav

Bank 2

- 250-SoundEffect_01.wav
- 260-SoundEffect_02.wav
- 299-HighVoice.wav

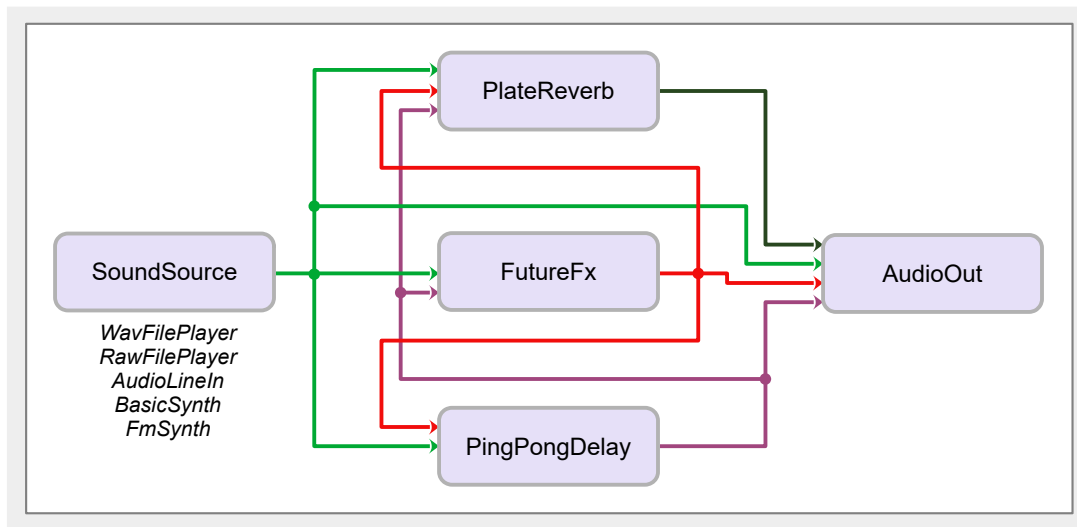
The same conventions are used for RAW audio file banks stored in non-volatile Serial Flash.

Filename Management

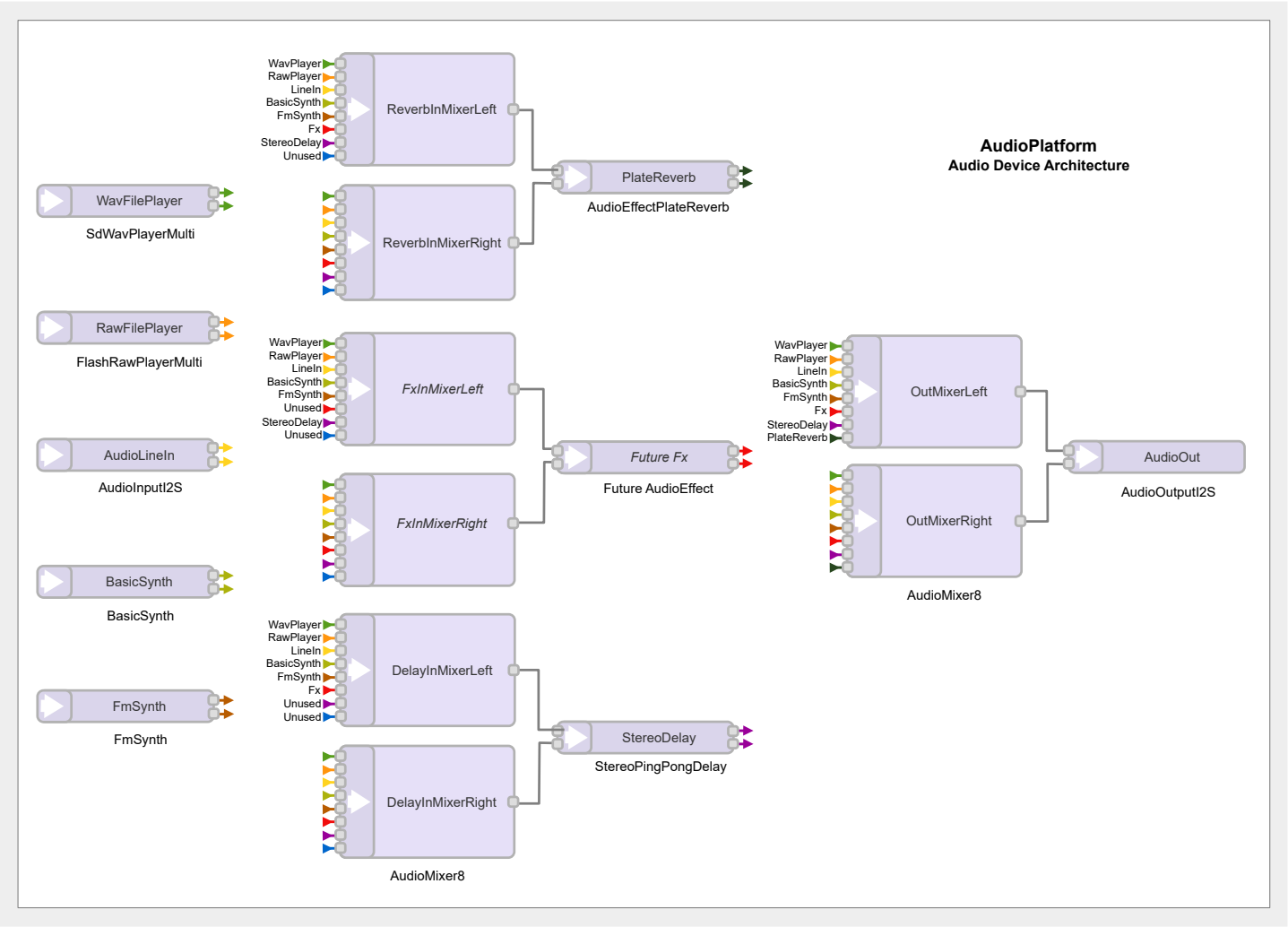
In general, the source of most user-supplied audio WAV and Raw files will be

Audio Architecture

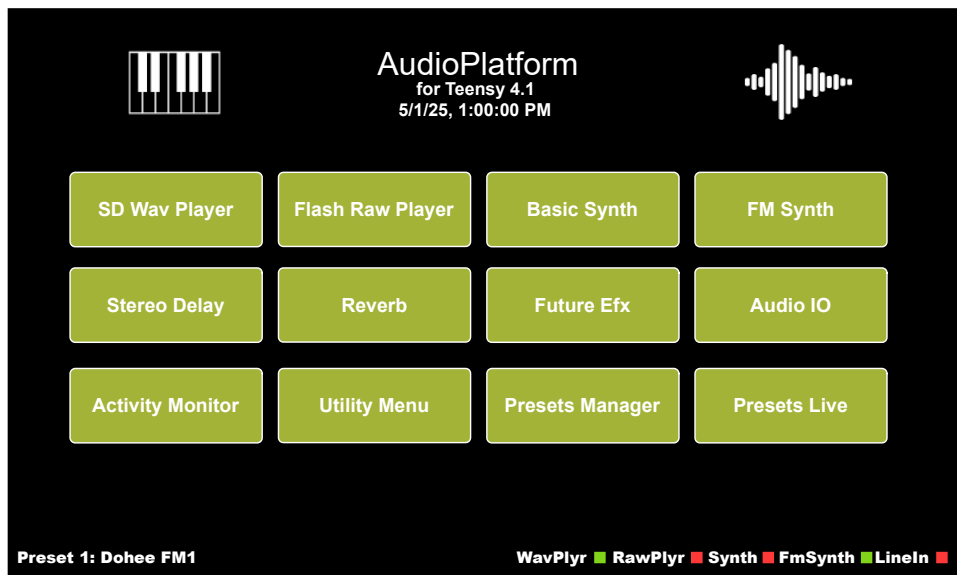
Signal Flow



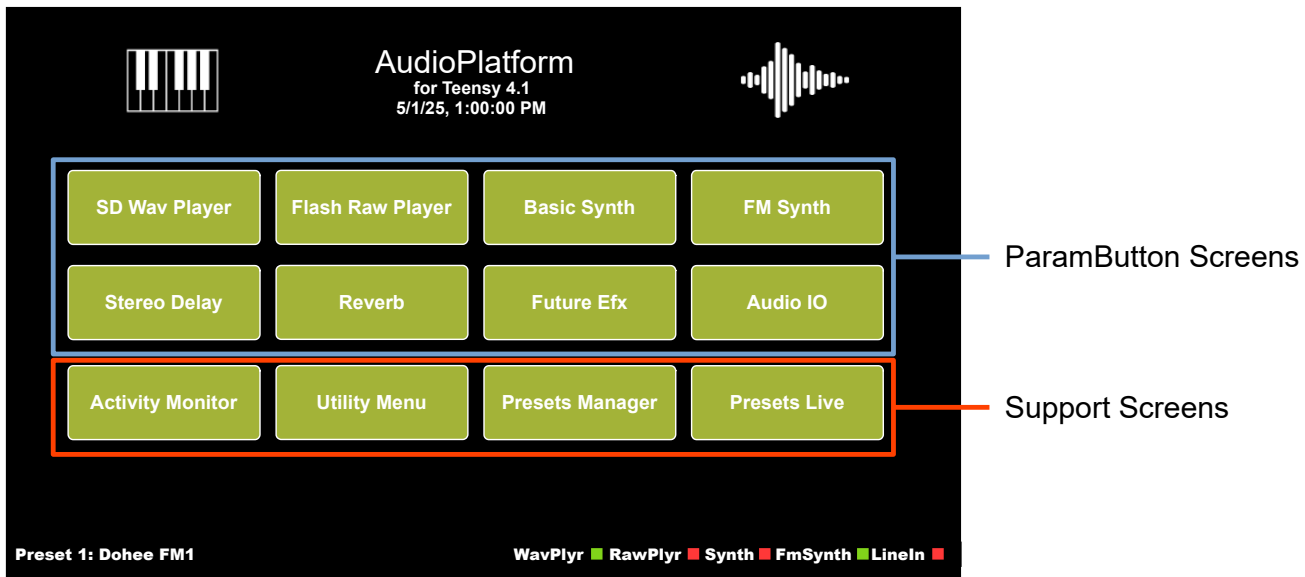
Connection Detail



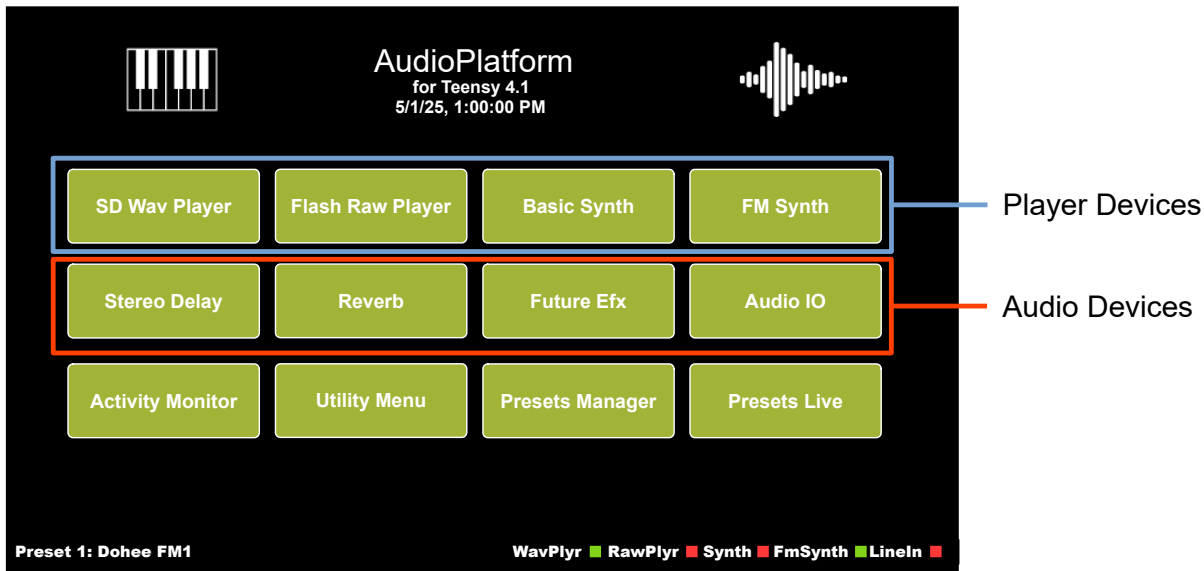
Main Menu



Main Menu Screen Types



Player and Audio Devices



Player Devices

Player Devices occupy the topmost row of the Main Menu screen. apApp features 4 players, which play audio media in response to inputs from an external MIDI device, and each of which operates on a separate media type:

- The *SD Wav Player* plays Stereo WAV formatted files located on the Teensy 4.1 SD card

- The *Flash Raw Player* plays Monaural (single channel, headerless) audio files located in the AudioPlatform's 128 MB non-volatile Flash Memory
- The *Basic Synth* is an implementation of a basic digital audio synthesizer with 12-voice polyphony
- The *FM Synth* is a Teensy Audio System implementation of the micro-Dexed synthesizer which is an emulator of the classic DX7 synthesizer, featuring 8-voice polyphony

Audio Devices

Audio Devices occupy the middle row of the Main Menu screen. apApp features 4 audio devices, which play various roles in the apApp audio architecture:

- The *Stereo Delay* device implements a stereo “ping-pong” digital delay, with up to 3 seconds of delay per channel, and both individual channel and cross-channel feedback paths
- The *Reverb / EFX* device features a Plate Reverberation unit, and a second effects unit pathway intended for addition of a 3rd audio effect in future apApp releases.
- The *Audio IO* device provides for control of the levels and effects settings of the system audio paths, with settings for Input and Output levels, effects send levels, and other audio system parameters
- The *MIDI* device features settings related to a connected MIDI control device, such as MIDI channel and destination player settings

Support and Utility Devices

Support Devices occupy the bottom row of the Main Menu screen, and serve as a gateway to additional screens and menus that perform various system and user support functions.

- The *Activity Monitor*
- The *Utility Menu*
- The *Presets Manager*
- The *Presets Live* device

Screens and Settings

Screen Icons

Screen icons (*buttons*) appear on all screens, and serve to allow the user to navigate among application screens (*Navigation Icons*), and to make selections or perform actions on the currently displayed screen (*Action Icons*).

Navigation Icons

Home



This button appears in the upper-right corner of all level 2 screens (screens one level below the main menu). Touching the button returns the user to the Main Menu screen.

Back



This button appears in the upper-right corner of level 3 and greater screens (screens at least two levels below the Main Screen).

Enter



This button may appear in various screens to provide access to a lower-level screen that is only accessible from the current screen.

Exit



This button appears in screens accessed using the Enter button, allowing the user to return to the screen which invoked the currently displayed screen.

Action Icons

All Notes Off



This button appears at the top of the Presets Live screen. When pressed, all notes currently playing are silenced. (Note: this function can also be assigned to a button or other control on a connected MIDI keyboard or controller).

Ignore Note Off



This button appears at the top of the Presets Live screen. When pressed, all notes currently held or played thereafter continue to play, ignoring key or sustain pedal releases. (Note: this function can also be assigned to a button or other control on a connected MIDI keyboard or controller).

Save Changes



This button appears at the top of all ParamButton screens. The text label of the button “Save Changes” normally appears in white. If any parameter changes are made to the displayed device using a rotary encoder, the text changes to red “**Save Changes**”, and remains so until the button is touched, at which point it returns to its normal white appearance. This process is discussed in detail in the **SECTION REFERENCE**.

Copy Settings



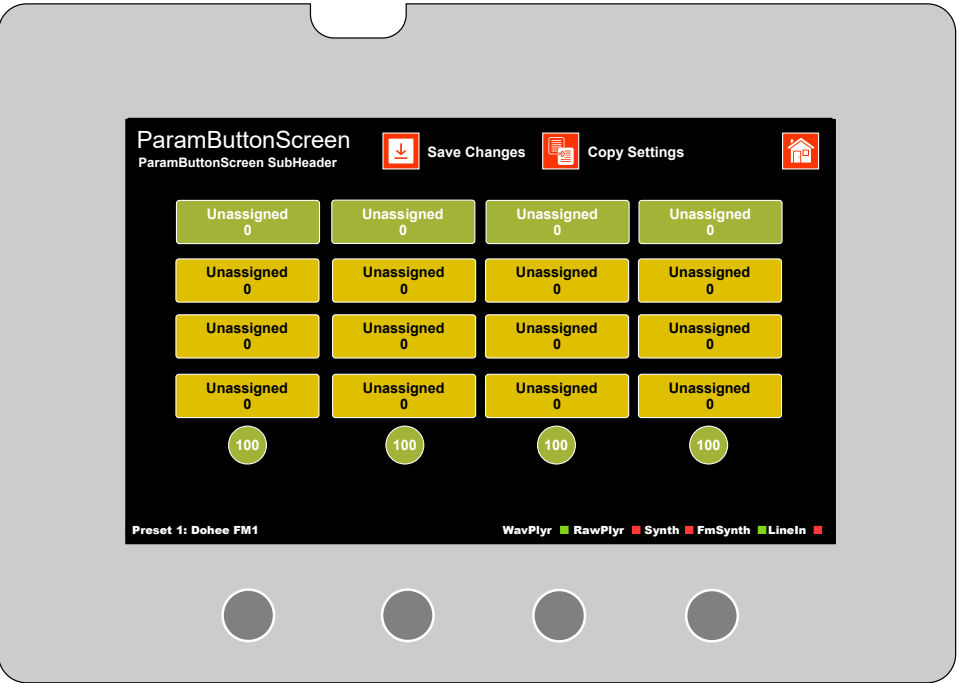
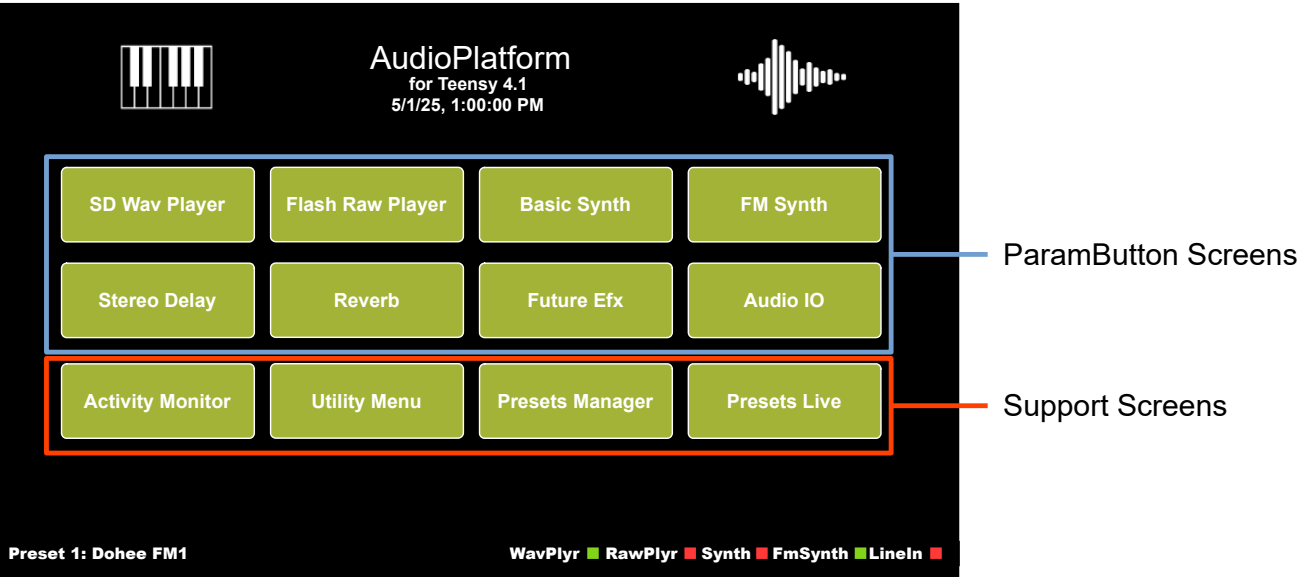
This button appears on all ParamButton screens next to the Save Changes button. When pressed, it displays a screen with buttons representing the presets currently loaded in the *Import* presets bank. When one of the import preset buttons is touched, the device settings of the import device are copied to the currently displayed device. This process is discussed in detail in the **SECTION REFERENCE**.

Scroll Up Down

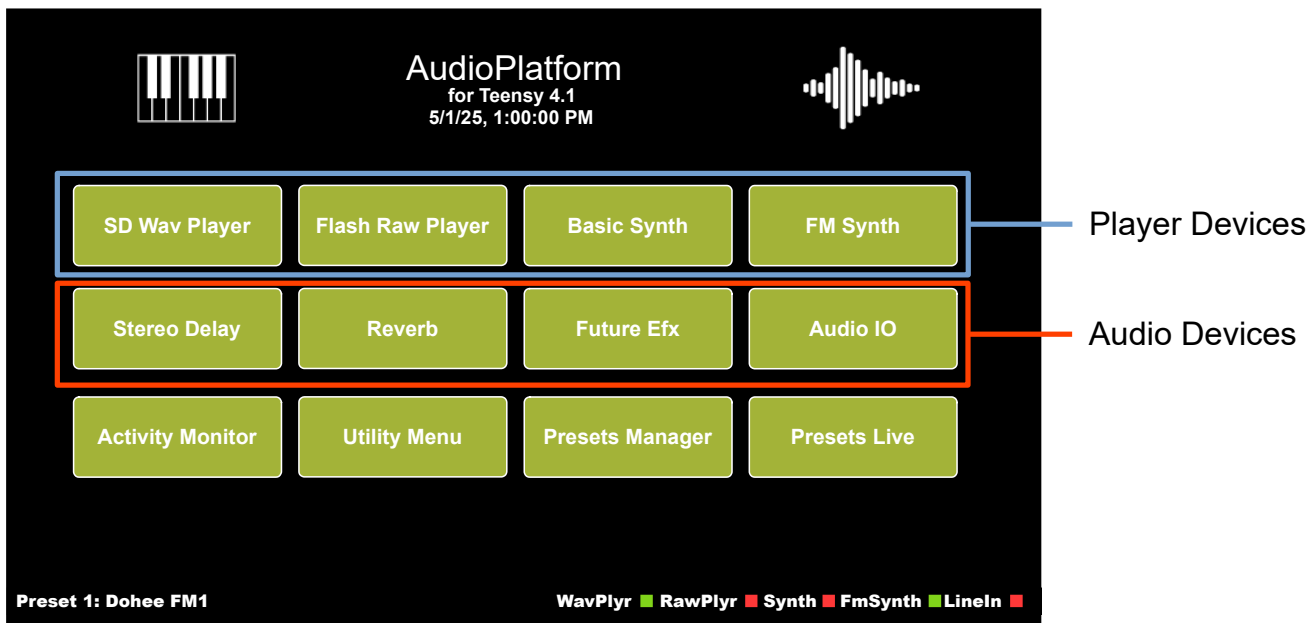


These buttons appear on screens that display lists of files or other names either for review or selection of items (for example, a preset bank file on the SD card). When simply displaying lists, the screen will scroll up or down repeatedly until the button is released. When displaying items for selection, repeated touches are required to scroll the list.

ParamButton Screens



Player Device Screens



Player Device Common Settings

- Player Status
- Player Level
- Player → Delay
- Player → Reverb
- Note Shift
- Note Min
- Note Max
- Pan Type
- Pan Parameter

PitchToMIDI Simulation for WAV and Raw Players

Simultaneous Playback

1. In Ableton, generate MIDI data for the WAV sample in question.
2. Use the MIDI data to create a second WAV sample that plays an instrument using the MIDI data.

3. Output both WAV samples to drive, with similar names.
4. Load the 2 WAV samples onto SD (or Serial Flash) and play the samples simultaneously from the MIDI keyboard to simulate synchronization.

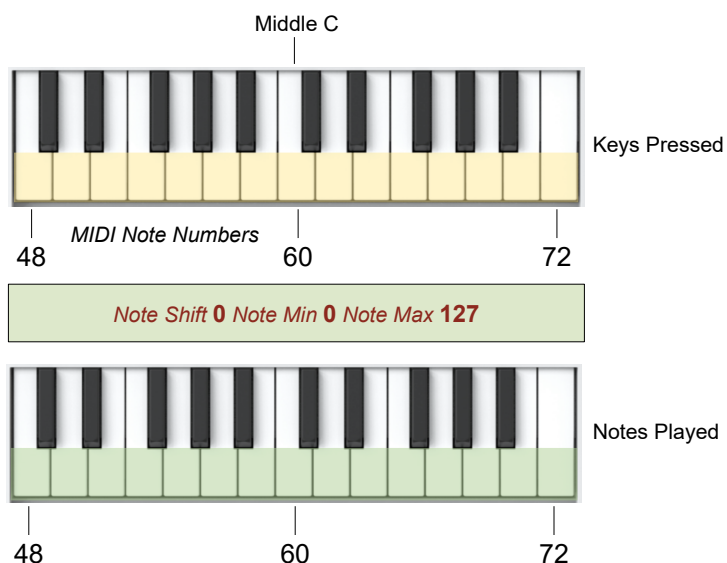
Variations

- Combine both original and MIDI sample in same stereo WAV file (hard LR, or mixed)
- (Best) Add a built-in Pitch to MIDI converter to AudioPlatform

Player Keyboard Mappings

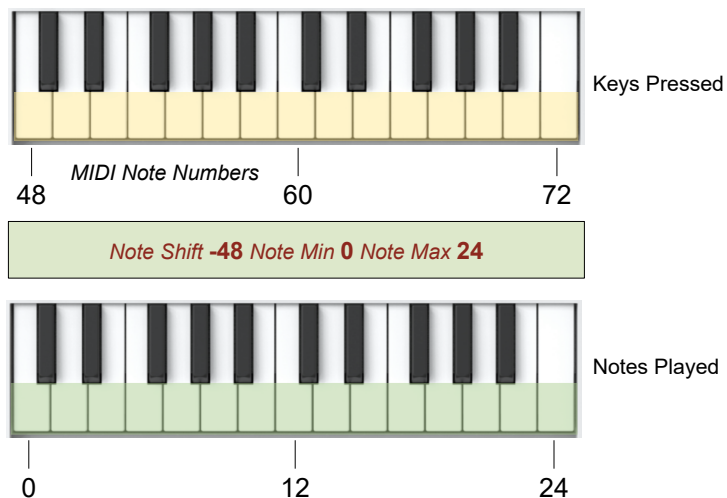
Among the device parameters of each Player Device are 3 settings that determine which keys on the MIDI keyboard will play which notes on the player's designated audio device.

Default Mapping

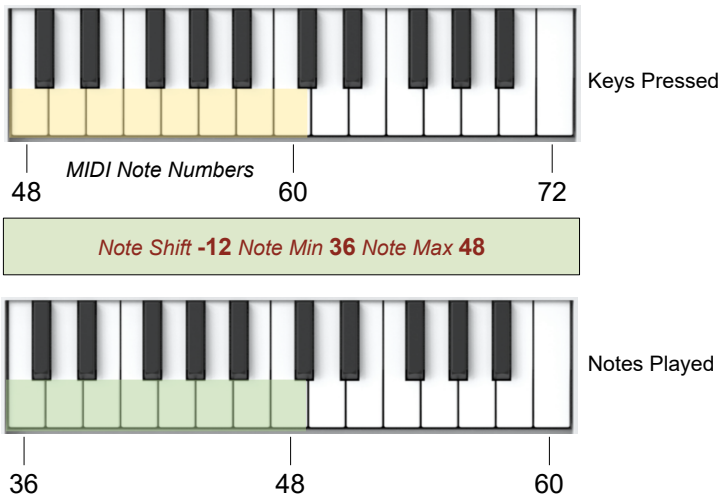


Shifted and Limited Mappings

All Notes Shifted Left 4 Octaves



12 Notes Shifted Left



WAV Player

WAV File Player
Teensy4.1 SD WAV Files

 Save Changes  Copy Settings 

Player Status Enabled	Player Level 0.0 dB	Player→Delay 58 %	Player→Reverb 80%
Sound Bank 0	Note Shift -12	Note Min 0	Note Max 99
Pan Type Center	Pan Parameter 63.0	Unassigned 0	Unassigned 0
1	0.5	1	1

Preset 1: Dohee FM1

WavPlyr  RawPlyr  Synth  FmSynth  LineIn 

RAW Player

Raw File Player
Serial Flash Raw Audio Files

 Save Changes  Copy Settings 

Player Status Disabled	Player Level -9.5 dB	Player→Delay 0 %	Player→Reverb 0%
Sound Bank 0	Note Shift 0	Note Min 0	Note Max 99
Pan Type Center	Pan Parameter 63.0	Unassigned 0	Unassigned 0
1	0.5	1	1

Preset 1: Dohee FM1

WavPlyr  RawPlyr  Synth  FmSynth  LineIn 

Basic Synthesizer

Basic Synthesizer

12 Voice Polyphony



Save Changes



Copy Settings



Synth Status Disabled	Synth Level 0.0 dB	Player→Delay 0 %	Player→Reverb 0%
Synth Type AM	Note Shift 0	Note Min 0	Note Max 99
Attack 0 ms	Decay 0 ms	Sustain 0 %	Release 0 ms
Mod Freq 0 Hz	Mod Amt 80 %	Pan Type Center	Pan Parameter 63.0
1	0.5	1	1

Preset 1: Dohee FM1

WavPlyr ■ RawPlyr ■ Synth ■ FmSynth ■ Lineln ■

AudioPlatform for Teensy 4.1

Application Reference

Page 40/82

FM Synthesizer

Basic Synthesizer
12 Voice Polyphony

 Save Changes  Copy Settings 

 Voices

Synth Status
Disabled

Synth Level
-8.5 dB

Player→Delay
20 %

Player→Reverb
60%

Voice Name
Lute

Note Shift
14

Note Min
0

Note Max
127

PitchBend Range
1 Semitones

Pan Type
Center

Pan Parameter
63.0

Unassigned

1

0.5

1

1

Preset 1: Dohee FM1 WavPlyr  RawPlyr  Synth  FmSynth  Lineln 

FM Synthesizer Voice Banks

FM Synth Voice Banks
Select Bank and Voice

Voice

BRASS 1

BRASS 2

BRASS 3

STRINGS 1

STRINGS 2

STRINGS 3

ORCHESTRA

PIANO 1

PIANO 2

PIANO 3

E.PIANO 1

GUITAR 1

GUITAR 2

SYN.LEAD 1

BASS 1

BASS 2

E.ORGAN 1

PIPES 1

HARPSICH 1

CLAV 1

VIBE 1

MARIMBA

KOTO

FLUTE 1

ORCH-CHIME

TUB BELLS

STEEL DRUM

TIMPANI

REFS WHISTL

VOICE 1

TRAIN

TAKE OFF

Voice Bank

rom1a

rom1b

rom2a

rom2b

rom3a

rom3b

rom4a

rom4b

Ritch1

Ritch2

User1

User2

User3

User4

User5

User6


FM Synth

FM Synthesizer Emulator

synth_dexed is a port of the Dexed sound engine (<https://github.com/asb2m10/dexed>) as library for the Teensy-3.5/3.6/4.x with an audio shield.

Dexed ist heavily based on <https://github.com/google/music-synthesizer-for-android>

synth_dexed.cpp/h used in AudioPlatform is from:

Arduino Resources\Third Party Code\microdexed-touch\third-party\Synth_Dexed\src\

PC/Mac Editor

<https://asb2m10.github.io/dexed/>



Audio Device Screens

Stereo Delay

Stereo Delay

Dual Channel X-Feedback

 Save Changes  Copy Settings 

Delay L Gain 0.0 dB	Delay L→Reverb 70%	Delay R Gain 0.0 dB	Delay R→Reverb 70%
Delay L Time 360 ms	Delay L FB % 35 %	Delay R Time 510 ms	Delay R FB % 35 %
Delay L xFB Time 90 ms	Delay L xFB % 19 %	Delay R xFB Time 75 ms	Delay L xFB % 10 %
0.5	1	0.5	1

Preset 1: Dohee FM1

WavPlyr  RawPlyr  Synth  FmSynth  LineIn 

Reverb

Reverb Settings

Future Additions TBD

 Save Changes  Copy Settings 

Reverb Size 80	Reverb Diffusion 70 %	Reverb Level 0.0 dB	UnAssigned 0
Unassigned 0	Unassigned 0	Unassigned 0	Unassigned 0
1	1	0.5	1

Preset 1: Dohee FM1

WavPlyr  RawPlyr  Synth  FmSynth  LineIn 

Future Efx

FutureEfx Settings

Audio Efx TBD

 Save Changes  Copy Settings 

Reverb Level 0.0 dB	Param2 % 25	Param3 % 25	Param4 % 25
Unassigned 0	Unassigned 0	Unassigned 0	Unassigned 0
1	1	1	1

Preset 1: Dohee FM1

WavPlyr  RawPlyr  Synth  FmSynth  LineIn 

Audio IO

Audio IO Settings

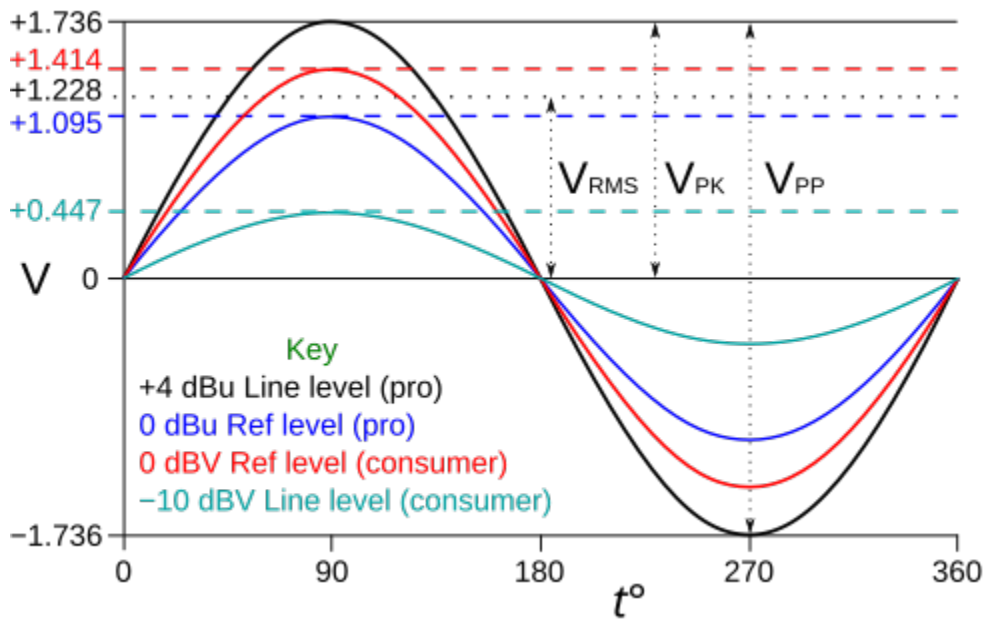
Audio In/Out/Codec

 Save Changes  Copy Settings 

LineIn Status Disabled	LineIn Level 1.33 Volts p-p	LineOut Level 2.83 Volts p-p	Phone Volume 80 %
LineIn→Delay 0 %	LineIn→Reverb 100 %	Unassigned 0	Unassigned 0
1	1	1	1

Preset 1: Dohee FM1

WavPlyr  RawPlyr  Synth  FmSynth  LineIn 



MIDI Settings and Controllers

MIDI Settings
For Future Development

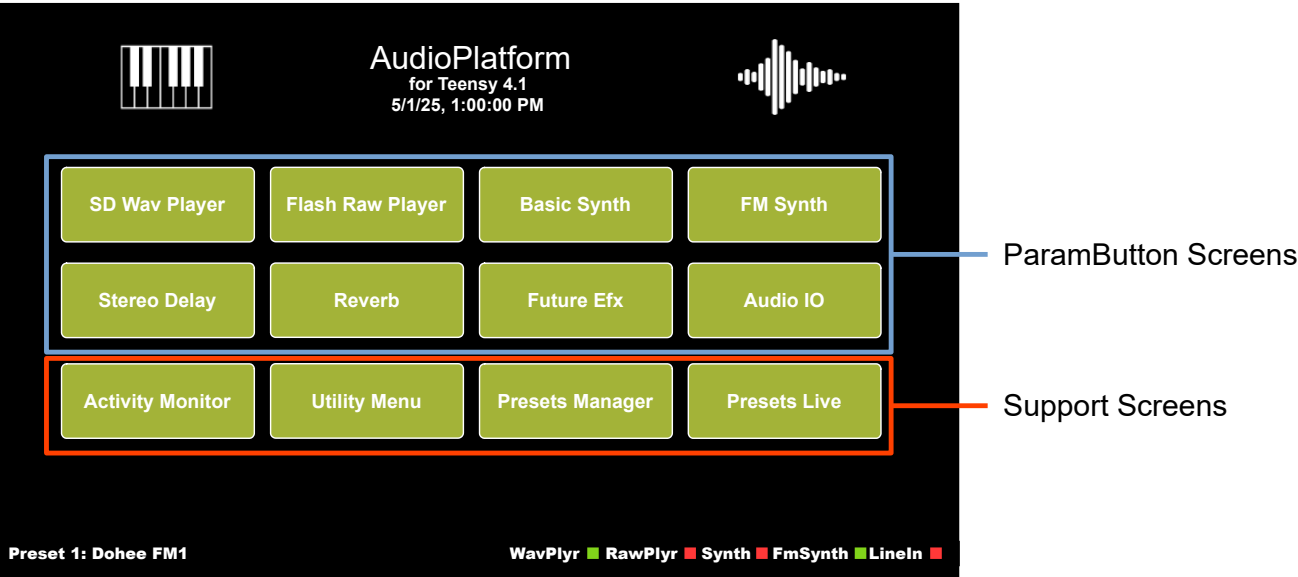
Save Changes
 Copy Settings

Wav Player Chan 0	Raw Player Chan 0	BasicSynth Chan 0	FmSynth Chan 0
MIDI In Device Generic	Unassigned 0	Unassigned 0	Unassigned 0
Unassigned 0	Unassigned 0	Unassigned 0	Unassigned 0
Unassigned 0	Unassigned 0	Unassigned 0	Unassigned 0

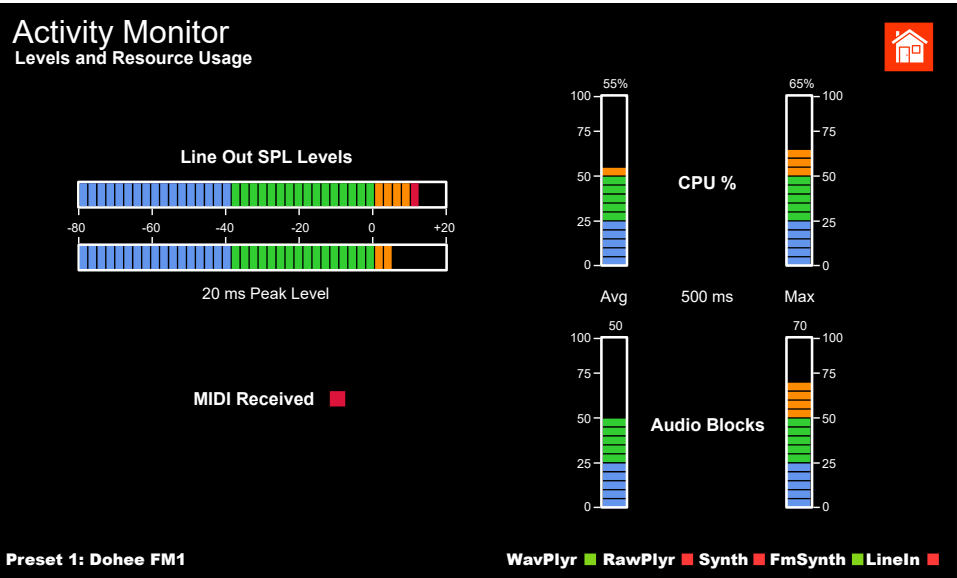
1
1
1
1

Preset 1: Dohee FM1
WavPlyr
RawPlyr
Synth
FmSynth
LineIn

Supprt Screens



Activity Monitor



Utility Menu

Utility Menu
System Management



System Profile

Media Viewer

Serial Flash

Set Time

Color Viewer

MIDI Settings

Test TextEdit

Test ListSelect

MIDI Settings

MIDI Settings
For Future Development

 Save Changes

 Copy Settings



Wav Player Chan
0

Raw Player Chan
0

BasicSynth Chan
0

FmSynth Chan
0

MIDI In Device
Generic

Unassigned
0

Unassigned
0

Unassigned
0

Unassigned
0

Unassigned
0

Unassigned
0

Unassigned
0

Unassigned
0

Unassigned
0

Unassigned
0

1

1

1

1

Preset 1: Dohee FM1

WavPlyr ■ RawPlyr ■ Synth ■ FmSynth ■ LineIn ■

Presets Manager

Presets Management

Load,Save,Edit Presets & Banks

Dohee FM1

Synth 1

Synth 2

Synth Rpt

Dohee Efx

Synth Rvb

Synth Rich

Woo Woo

1

2

3

4

5

6

7

8

MicroTone L

Synth Pulse

MicroTone S

Synth Marim

WAV Plyr

RAW Plyr

Basic Synth

FM Cluster

9

10

11

12

13

14

15

16

FM Synth

Dohee FM2

Dohee Shift

Multi 1

Synth Rpt 2

Guitar

17

18

19

20

21

22

23

24

Synth Pulse

Cluster

FM Piano

ALL OFF

25

26

27

28

29

30

31

32

Preset: MicroTone L

Active Bank: DefaultBank

Press Load, Save, Import, Clear, or Rename

Press Load or Save

Load

Save

Import

Clear

Rename

Cancel

Load

Save

Default

SD Card

Cancel

Presets Management

Load,Save,Edit Presets & Banks

Dohee FM1

Synth 1

Synth 2

Synth Rpt

Dohee Efx

Synth Rvb

Synth Rich

Woo Woo

1

2

3

4

5

6

7

8

MicroTone L

Synth Pulse

MicroTone S

Synth Marim

WAV Plyr

RAW Plyr

Basic Synth

FM Cluster

9

10

11

12

13

14

15

16

FM Synth

Dohee FM2

Dohee Shift

Multi 1

Synth Rpt 2

Guitar

17

18

19

20

21

22

23

24

Synth Pulse

Cluster

FM Piano

ALL OFF

25

26

27

28

29

30

31

32

Preset: MicroTone L

Active Bank: DefaultBank

Press Load, Save, Import, Clear, or Rename

Press Load or Save

Load

Save

Import

Clear

Rename

Cancel

Load

Save

Default

SD Card

Cancel

Presets Live

Presets Live

Direct Preset & Bank Access

All Notes Off

Ignore Note Off

1 Dohee FM1	2 Synth 1	3 Synth 2	4 Synth Rpt
5 Dohee Efx	6 Synth Rvb	7 Synth Rich	8 Woo Woo
9 MicroTone L	10 Synth Pulse	11 MicroTone S	12 Synth Marim
13 WAV Plyr	14 Raw Plyr	15 Basic Synth	16 FM Cluster
17 FM Synth	18 Dohee FM2	19 Dohee Shift	20 Multi 1
21	22 Synth Rpt 2	23 Guitar	24
25 Synth Pulse	26 Cluster	27 FM Piano	28
29	30	31	32 ALL OFF

DefaultBank.pbk

DefaultBank-250220.pbk
DefaultBank-250319.pbk
DefaultBank-250415.pbk
DefaultBank-250519.pbk
ImportActiveTest.pbk
SynthMicroToneCmnDly.pbk
FmSynthPanTest.pbk
MultiPlayerTest1.pbk
MultiPlayerTest2.pbk
WavPlayerMultiEffects.pbk
BasicSynthMultiEffects.pbk

Active Bank: DefaultBank.pbk

Preset Banks on SD

Presets 1: Dohee FM1

WavPlyr RawPlyr Synth FmSynth LineIn

TeensyDuino

TeensyDuino 1.58.1

Modifications to 1.58.1 Modules

Arduino IDE

Arduino IDE 2.0 Build Failures / Inconsistent State

Repeated build failures, even when going back to Zipped versions from previous dates which are known to have been building and running successfully were seen on 10/27/23 when building

```
D:\SwHwDev\Arduino_Proto_Dev\Projects\Teensy_4.1_Proto_Board\Test_Sketches\PlaySD_Interactive
```

which had been building successfully on a 10/26/23. In editing sessions on 10/27, which made no changes to this directory or on any of the applications dependencies, failed with various errors reported. It seems that certain modules that had been modified were not being re-compiled, such that the previous versions were being used, causing errors about, for example, undefined objects, as shown in this line from the build output (not necessarily the object causing the issues:

```
Using previously compiled file:
```

```
C:\Users\Donald
```

```
Swearingen\AppData\Local\Temp\arduino\sketches\9D6A1A37B3B4F5D6A355A2CEF7E17CBB\...
```

It turns out that the folder listed here is an example of the location where the 2.0 compiler places all the compiled objects and related files, including the .hex file for the application (which can be copied and saved for re-use by the Teensy Loader application. The solution to the build errors turned out to be deleting this folder so that the compiler is forced to perform a clean build of all the modules required in the application.



Note that new subdirectories (with long hex names, as shown above) are added to the sketches folder on successive builds and can be safely deleted (when the Arduino IDE is not running).

Note on C++ Dynamic Casting

From <https://www.ibm.com/docs/en/i/7.3?topic=expressions-dynamic-cast-operator-c-only>

```
#include <iostream>
using namespace std;

struct A {
    virtual void f() { cout << "Class A" << endl; }
};

struct B : A {
    virtual void f() { cout << "Class B" << endl; }
};

struct C : A {
    virtual void f() { cout << "Class C" << endl; }
};

void f(A* arg) {
    B* bp = dynamic_cast<B*>(arg);
    C* cp = dynamic_cast<C*>(arg);

    if (bp)
        bp->f();
    else if (cp)
        cp->f();
    else
        arg->f();
};

int main() {
    A aobj;
    C cobj;
    A* ap = &cobj;
    A* ap2 = &aobj;
    f(ap);
    f(ap2);
}
```

If the above scheme is used in Arduino IDE 2.0, as in:

```
.h:

class ParamDisplayScreen : public DisplayScreen {

// Constructor/Destructor
//
public:
    ParamDisplayScreen();
    virtual ~ParamDisplayScreen();
```

```

        .
        .
        .
    void dummyFn();
}

.ino:

ParamDisplayScreen paramScreen;
DisplayScreen* currentScreen;
.
.
.
currentScreen = &paramScreen;
if (dynamic_cast<ParamDisplayScreen*>(currentScreen)) {
    Serial.printf("currentScreen is ParamDisplayScreen\n");
    dynamic_cast<ParamDisplayScreen*>(currentScreen)->dummyFn();
}

```

The compiler generates the error: Compilation error: 'dynamic_cast' not permitted with '-fno-rtti'

This is discussed at <https://arduino.stackexchange.com/questions/55157/dynamic-cast-not-permitted-with-fno-rtti> which indicates:

It seems avr-gcc version 4.9.2 that is used by the Arduino IDE has been compiled without RTTI support. It defaults to -fno-rtti, and if you try turning it on with -frtti you just get other horrible linker error messages instead.

So it looks like you can't use dynamic_cast on AVR boards.

Set Pointers to Functions Within Class in Main

```

#include <iostream>
using namespace std;
class AB {
public:
    int sub(int a, int b) {
        return a-b;
    }
    int div(int a, int b) {
        return a/b;
    }
};

//using function pointer
int res1(int m, int n, AB* obj, int(AB::*fp)(int,int)) {
    return (obj->*fp)(m,n);
}

//using function pointer
int res2(int m, int n, AB* obj, int(AB::*fp2)(int,int)) {
    return (obj->*fp2)(m,n);
}

```

```
}  
int main() {  
    AB ob;  
    cout << "Subtraction is = " << res1(8,5, &ob, &AB::sub) << endl;  
    cout << "Division is = " << res2(4,2, &ob, &AB::div) << endl;  
    return 0;  
}
```

Audio Library

AudioMemory

During initialization (*setup()*) memory for Audio Library operations is allocated using the code macro `AudioMemory(num)`. The argument `num` to the macro specifies the number of memory blocks (256+4 = 260 bytes) of memory to reserve for audio processing.

For example, the statement

```
AudioMemory(10)
```

allocates 2600 bytes in RAM.

The `AudioMemory()` macro is defined in Audio Library module `AudioStream.h` as follows:

```
#define AudioMemory(num) ({ \
    static DMAMEM audio_block_t data[num]; \
    AudioStream::initialize_memory(data, num); \
})
```

Audio Memory in DMAMEM

By default, the memory is allocated in DMAMEM, a segment of internal RAM memory of Teensy 4.1.

The argument "num" to the `AudioMemory(n)` macro indicates the number memory blocks to be reserved for audio usage.

At 44.1 kHz sample rate, 2 bytes per sample, 88.2 kBytes per second, a (256 + 4) byte "block" represents

$$260 / 88200 \approx 2.9 \text{ ms}$$

of sample memory (the extra 4 bytes are for accounting purposes). In particular, for a digital delay of maximum length 2 seconds, the required number of blocks would be:

$$2.0 / (260.0 / 88200.0) \approx 678 \text{ blocks} = 176,280 \text{ bytes of RAM.}$$

The maximum number of bytes in internal RAM available for audio memory, `MAX_AUDIO_MEMORY`, is defined in module `AudioStream.cpp` as 229,376, or ~882 blocks. So a delay of 2 seconds is possible, with room left for other memory usage by other Audio Library devices. So we calculate the argument to the `AudioMemory` macro accordingly, adding an additional number of blocks to ensure that other operations are supported.

Audio Memory in EXTMEM

If external PSRAM is available, it has been seen that `AudioStream.h` and `AudioStream.cpp` can be modified to substitute EXTMEM for allocation of Audio Memory; or a second region, separate from

DMAMEM, can be defined for audio memory used for specific audio devices and uses, while DMAMEM remains the source for the AudioMemory macro. See the Teensy Audio Forum discussions:

<https://forum.pjrc.com/index.php?threads/use-extmem-psram-16mb-soldered-onto-teensy-4-1-pads-for-audiomemory.63241/>

and

<https://forum.pjrc.com/index.php?threads/malloc-free-for-extmem-and-dtcm.63695/>

AudioConnection

Once a connection (patchcord) has been made to a specific device input, successive attempts to connect to the same input are (silently) rejected until the first connection is removed by calling `patchcord.remove()`.

Update: This is because only a single connection to an input is allowed, whereas outputs can accommodate multiple connections.

AudioMixer4 in SubClass

All attempts to make connections to AudioMixer4 instances of a class within the class itself (sdwavplayer->mixer->gain, for example) seem to fail. Connections among other Audio objects (sdwavplayer->envelope->gain, for example) seem to work correctly. However, class objects can be successfully connected to mixers defined in the owner (instantiator) class and passed via the constructor of the instance.

Update: Apparently, this was due to early attempts to define connections.

Non-Static AudioConnections

For non-static audio connectors, connections must be made in the class constructor using the `connect()` method:

.h file

```
Class xxx {  
    AudioConnection patchcord;  
}
```

.cpp file

```
xxx::xxx {  
    patchcord.connect(outDevice, outDeviceChan, inDevice, inDeviceChan);  
}
```


Static AudioConnections

For static audio connectors, connections must be made in either the static definition preceding the class, or instantiated up the prelude and connected in the constructor using the *connect()* method

.h file

```
Class xxx {  
    static AudioConnection patchcord;  
}
```

.cpp file

```
AudioConnection xxx::patchcord(outDevice, outDeviceChan, inDevice, inDeviceChan);
```

```
xxx::xxx {  
}
```

or

.h file

```
Class xxx {  
    static AudioConnection patchcord;  
}
```

.cpp file

```
AudioConnection xxx::patchcord;
```

```
xxx::xxx {  
    patchcord(outDevice, outDeviceChan, inDevice, inDeviceChan);  
}
```

TeensyDuino Library Installation, Additions and Modifications

AudioPlatform 1.0 utilizes a number of additions and/or modifications to the most recent 1.59.0 version of TeensyDuino library. These changes to the 1.59.0 version are detailed and described in the following entries. The source code for the additions is included in the AudioPlatform distribution, along with detailed instructions as to how to do so.

Installing TeensyDuino

- Where to download
- Where located
- How to install additions

TeensyDuino Modifications and Additions for AudioPlatform

AudioEffectDelayExternal

Code by Teensy Audio Forum user:

<https://forum.pjrc.com/index.php?members/h4yn0nnym0u5e.82492/>

Downloaded code from:

<https://github.com/h4yn0nnym0u5e/Audio/tree/feature/AudioEffectDelayExternal>

ZIP appears to also include an entire Teensy core Audio Library from a version of Teensy previous to 1.59.0. Only the following files copied from the downloaded ZIP directory to installed Teensy 1.59.0 Audio library:

- **effect_delay.ext.cpp**, **effect_delay_ext.h** (updated version supports delay memory in EXTMEM)
- **extmem.cpp**, **extmem.h** (not part of Teensy 1.59.0; copied to installed library for support of updated effect_delay_ext.cpp/h)

MediaPlayers

Custom player classes for playback of audio files from the microSD card and Serial Flash.

<https://github.com/FrankBoesing/Teensy-WavePlayer>

play_wav.cpp, play_wav.h: Advantages and problems

According to Ken Hahn: Frank B (MCU32) doesn't do much with Teensy anymore and I wouldn't expect him to do any library updates on his old Teensy stuff. He wasn't happy that his PRs weren't getting rolled into Teensyduino as fast as he thought they should be, packed up his toys and pretty much left the forum.

Ken suggests looking at H4yn0nnym0u5e's new player:

<https://forum.pjrc.com/index.php?threads/yet-another-file-player-and-recorder.70963/>

Latest:

<https://forum.pjrc.com/index.php?threads/yet-another-file-player-and-recorder.70963/page-9>

I downloaded latest git “feature” from

<https://github.com/h4yn0nnym0u5e/Audio/tree/feature/buffered-SD-audio-EvR>

And placed in 3rd party folder of Arduino Resources.

On further examination, however, I realized that attempting to use his code was going to be far more than a drop-in effort. Not only would it require numerous changes to existing code, additions of new files, etc.; but the most challenging aspect is that I'd need to modify the fundamental structures of my existing SD players: when using the standard Teensyduino SD WAV player, and also Frank B's `play_wav`, a single instance of the player can handle additional play file commands, such that multiple notes can be played by the single player. In H4yn0nnym0u5e's (I still can't figure out how to pronounce that name; maybe it's not supposed to be pronounced?) buffered player, one must allocate an instance of the class for every voice: if you want to play 8 notes simultaneously, you need 8 instances of the class. This totally invalidates my current and very smoothly working voice management/replacement scheme.

Bottom line: I will check out his example sketches when I have time in order to verify that this all actually works. If so, I'll put it on my to do list. But for now, there simply is no time to stop everything else and spend who knows how much time to add this feature in order to work around the limitations of the Teensy SD and `play_wav` (note that users can select which one as a compile option, depending on which problems they are most comfortable with accommodating).

ILI9341 Display Support

The ILI9341 driver code included in the TeensyDuino 1.59.0 release, is removed from the installation library. The ILI9341 driver is not used in AudioPlatform, but there is still a dependency on some of the fonts defined for the ILI9341. These references are satisfied by a version of ILI9341.h that is included in the Arduino *Libraries* folder of the AudioPlatform distribution.

RA8875 Display Support

AudioPlatform uses the RA8875 LCD touchscreen display. But for successful operation, there are several modifications to the RA8875 folder of the TeensyDuino 1.59.0 distribution:

Folder RA8875/src/settings/RA8875UserSettings.h:

Enable capacitive touch screen by “uncommenting” line 63:

Before

```
//#define USE_FT5206_TOUCH//capacitive touch screen
```

After

```
#define USE_FT5206_TOUCH//capacitive touch screen
```

SD WAV Player

Code by Teensy Audio Forum user:

f.boesing@gmx.de

Downloaded code from:

<https://github.com/FrankBoesing/Teensy-WavePlayer/tree/5cc77153067f907a4b121435028ede3f47878173>

Copied 2 files from above to installed Teensy 1.59.0 Audio library:

- **play_wav.cpp, play_wav.h**

Defines improved SD WAV file player AudioPlayWav (play_wav.h) that uses far less CPU% than original AudioPlaySdWav (play_sd_wav.h) while adding the ability to also play other formats, including multichannel WAV files.

For proper operation, users must also make an addition to the standard 1.59.0 file Audiio.h:

Modify the lines:

```
#include "play_sd_wav.h"
#include "play_serialflash_raw.h"
```

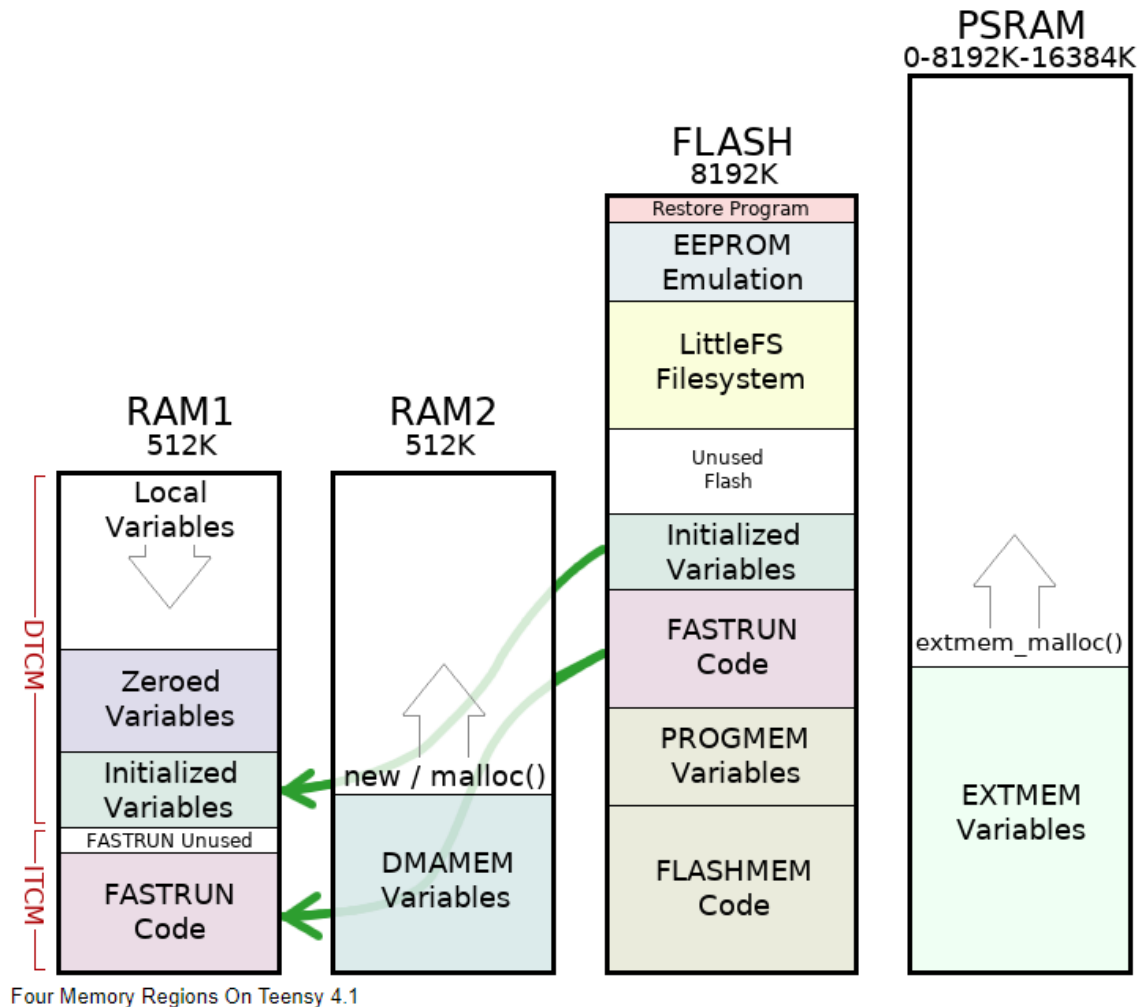
to add a reference to play_wav.h:

```
#include "play_sd_wav.h"
#include "play_wav.h"
#include "play_serialflash_raw.h"
```



As of 3/1/25 (approaching release 1.0 of AudioPlatform), there seems to be an issue with playback of mono WAV files that can crash the application at times. Until this is resolved, it is recommended that only stereo WAV files be played from the SD card.

Teensy 4.1 Memory Organization



Program / Flash Memory

Teensy 4.1 has 8 Mbyte of flash memory intended for storing your code. The flash memory can also store read-only variables and arrays. A portion of the flash memory may be used for file storage using the LittleFS library. The top 256K of this memory is reserved for EEPROM emulation data and the LED blink restore program.

RAM

1024K of memory is available for variables and data. Half of this memory (RAM1) is accessed as Tightly Coupled Memory (Data TCM, Instruction TCM) for maximum performance. The other half

(RAM2) is optimized for access by DMA. Normally large arrays & data buffers are placed in RAM2, to save the ultra-fast RAM1 for normal variables.

EEPROM

4284 bytes of emulated EEPROM memory is supported. Writing to this memory temporarily stalls code execution from flash. The EEPROM library is typically used to access this memory. AVR libc functions may also be used.

QSPI Memory Expansion

Teensy 4.1 has 2 locations to add 8 pin QSPI memory chips. Both locations support 8MB PSRAM chips. If only 1 PSRAM chip is used, it must be soldered to the smaller pads. The larger pads may be used with certain flash memory chips supported by the LittleFS library.

Ken: Since all systems will ship with 16MB PSRAM installed, should probably just call that out where appropriate

Static Allocation Keywords

When the compiler builds your program, all global variables, static variables, and compiled code is assigned to dedicated locations in memory. This is called static allocation, because the memory addresses are fixed. By default, allocation tries to use the ultra-fast DTCM & ITCM memory. The following keywords allow control over where the compiler will place your variables and code within the memory.

DMAMEM - Variables defined with DMAMEM are placed at the beginning of RAM2. Normally buffers and large arrays are placed here. These variables can not be initialized, your program must write their initial values, if needed.

EXTMEM - Variables defined with EXTMEM are placed in the optional PSRAM memory chip soldered to the QSPI memory expansion area on bottom side of Teensy 4.1. These variables can not be initialized, your program must write their initial values, if needed.

Note also that external PSRAM is located at address 0x70000000. Variables and structures whose definition is preceded by the EXTMEM keyword, will be located starting at this address.

PROGMEM & F() - Variables defined with PROGMEM, and strings surrounded by F() are placed only in the flash memory. They can be accessed normally, special functions normally used on 8 bit boards are not required to read PROGMEM variables.

FASTRUN - Functions defined with "FASTRUN" are allocated in the beginning of RAM1. A copy is also stored in Flash and copied to RAM1 at startup. These functions are accessed by the Cortex-M7 ITCM (Instruction Tightly-Coupled Memory) bus, for the fastest possible performance. By default, functions without any memory type defined are treated as FASTRUN. A small amount of memory is typically unused, because the ITCM bus must access a memory region which is a multiple of 32K.

FLASHMEM - Functions defined with "FLASHMEM" executed directly from Flash. If the Cortex-M7 cache is not already holding a copy of the function, a delay results while the Flash memory is read into the M7's cache. FLASHMEM should be used on startup code and other functions where speed is not important.

Dynamic Allocation

As your program runs, it may use all of the RAM which was not reserved by static allocation. Because the specific memory address for each variable is computed as your program runs, this is called dynamic memory allocation.

Local Variables - Local variables, and also return addresses from function calls and the saved state from interrupts are placed on a stack which starts from the top of RAM1 and grown downward. The amount of space for local variable is the portion of RAM1 not used by FASTRUN code and the initialized and zeroed variables.

Heap - Memory allocated by C++ "new" and C *malloc()*, and Arduino String variables are placed in RAM2, starting immediately after the DMAMEM variables.

External Heap - If PSRAM has been added, *extmem_malloc()* may be used to allocate this memory, started immediately after the EXTMEM variables. When no PSRAM is present, *extmem_malloc()* automatically allocated memory from the normal heap in RAM2.

RTC RAM

16 bytes of memory are located within the RTC. If a coin cell is connected to VBAT, contents of this memory is preserved while power is off. This memory is accessed as 32 bit registers LPGPR0-LPGPR3.

SD Card

A built in SD socket allows you to use SD cards for large data storage. The Arduino SD library is used to access the card, using `SD.begin(BUILTIN_SDCARD)`. This built in SD socket uses fast 4 bit native SDIO to access the card. SD cards may also be used via the SPI pins, with `SD.begin(cspin)`, using the slower single bit SPI protocol to access the card.

SPI Flash

Flash memory chips may be added using the SPI pins. These are supported by the SerialFlash and LittleFS libraries.

Ken: Should mention that there is a 1GB NOR Flash installed on the baseboard

Hardware Notes

NXP IMXRT1060

System Memory

Start Address	End Address	Size	Description
E010_0000	FFFF_FFFF	511MB	Reserved
E000_0000	E00F_FFFF	1MB	CM7PPB
8000_0000	DFFF_FFFF	1.5GB	SEMC external memories (SDRAM, NOR, PSRAM, NAND and 8080) shared memory space
7FC0_0000	7FFF_FFFF	4MB	FlexSPIRX FIFO
7F80_0000	7FBF_FFFF	4MB	FlexSPITX FIFO
7F40_0000	7F7F_FFFF	4MB	FlexSPI2 RXFIFO
7F00_0000	7F3F_FFFF	4MB	FlexSPI2 TXFIFO
7000_0000	7EFF_FFFF	240MB	FlexSPI2/FlexSPI2 ciphertext
6000_0000	6FFF_FFFF	256MB	FlexSPI/FlexSPICiphertext
4800_0000	5FFF_FFFF	384MB	Reserved
4400_0000	47FF_FFFF	64MB	Reserved
4210_0000	43FF_FFFF	31MB	Reserved
4200_0000	420F_FFFF	1MB	AIPS-5
4180_0000	41FF_FFFF	8MB	Reserved
4170_0000	417F_FFFF	1MB	GPVReserved
4160_0000	416F_FFFF	1MB	GPVReserved
4150_0000	415F_FFFF	1MB	GPVReserved
4140_0000	414F_FFFF	1MB	cpu configuration port
4130_0000	413F_FFFF	1MB	Reserved for ems GPV
4120_0000	412F_FFFF	1MB	Reserved for per GPV
4110_0000	411F_FFFF	1MB	m configurationport
4100_0000	410F_FFFF	1MB	main configuration port
4040_0000	40FF_FFFF	12MB	Reserved
4030_0000	403F_FFFF	1MB	AIPS-4
4020_0000	402F_FFFF	1MB	AIPS-3
4010_0000	401F_FFFF	1MB	AIPS-2
4000_0000	400F_FFFF	1MB	AIPS-1
3000_0000	3FFF_FFFF	256MB	Reserved
2040_0000	2FFF_FFFF	252MB	Reserved
2030_0000	203F_FFFF	512KB	OCRAM Reserved
2028_0000	202F_FFFF	512KB	OCRAM-0FlexRAM
2020_0000	2027_FFFF	512KB	OCRAM2
2010_0000	201F_FFFF	1MB	Reserved
2008_0000	200F_FFFF	512KB	DTCM Reserved
2000_0000	2007_FFFF	512KB	DTCM
1000_0000	1FFF_FFFF	256MB	Reserved
0800_0000	0FFF_FFFF	128MB	Reserved
0040_0000	07FF_FFFF	124MB	Reserved
0028_0000	003F_FFFF	1536KB	Reserved
0022_0000	0027_FFFF	384KB	ROMCP Reserved
0020_0000	0021_FFFF	128KB	ROMCP
0010_0000	001F_FFFF	1MB	ITCM Reserved
0008_0000	000F_FFFF	512KB	ITCM Reserved
0000_0000	0007_FFFF	512KB	ITCM

Teensy 4.1 Pin Usage

AudioPlatform for Teensy 4.1 Pin Assignments

	Color Key	PWM	Digital Audio	Serial	Digital		Analog	I2C	SPI	CAN Bus	Power			
Baseboard												Baseboard		
Power						GND						VIN	5V Power	
MIDI IN	RX1	PWM	CRX2	CS1	RX1	0	GND						GND	Ground
MIDI OUT	TX1	PWM	CTX2	MISO1	TX1	1	3.3V							No Connection
Touchscreen						INT	PWM	OUT2					MCLK1	T4.x Audio Adapter
Rotary Encoder 1 B						CLK_A	PWM	LRCLK2				PWM		I/O Pin 11
Rotary Encoder 1 Y						DT_B	PWM	BCLK2					BCLK	T4.x Audio Adapter
RA8875 TFT Display						CS	PWM	IN2					LRCLK	T4.x Audio Adapter
Flash 1 CS						CS	PWM	OUT1D					SCL	T4.x Audio Adapter/Touch
T4.x Audio Adapter						DIN	PWM	OUT1A	RX2				SDA	T4.x Audio Adapter/Touch
T4.x Audio Adapter						DOUT	PWM	IN1	TX2					
RA8875 TFT Display						RESET	PWM	OUT1C						
I/O Pin 1							PWM	MQSR	CS					
Flash MOSI						MOSI	PWM	CTX1	MOSI					
Flash MISO						MISO	PWM	MQSL	MISO					
No Connection												3.3V		
Rotary Encoder 1 G						SWITCH	PWM	SCL2	TX6	A10	24			
Rotary Encoder 2 B						CLK_A	PWM	SDA2	RX6	A11	25			
RA8875 TFT Display						MOSI1			MOSI1	A12	26			
RA8875 TFT Display						SCK1			SCK1	A13	27			
I/O Pin 10							PWM		RX7					
I/O Pin 8							PWM		TX7					
Rotary Encoder 2 Y						DT_B		CRX3			30			
Rotary Encoder 2 G						SWITCH		CTX3			31			
I/O Pin 6								OUT1B			32			
										</				

GPIO Expansion

				PWM	PWM					
	A17	A16	OUT1B	TX7	RX7					Teensy Pins
	41	40	32	29	28	3.3V	Gnd			
Top	2	4	6	8	10	12	14			GPIO Pins
Bottom	1	3	5	7	9	11	13			
	10	14	15	16	17	22	Gnd			Teensy Pins
	CS	A0	A1	A2	A3	A8				
	MQSR	TX3	RX3	RX4	TX4	CTX1				
	PWM	SPDIF OUT	SPDIF IN	SCL1	SDA1	PWM				
		PWM	PWM							

Color Key
Digital
PWM
Serial
Digital Audio
Analog
I2C
SPI
CAN Bus
Power

Rotary Encoders

Analog Potentiometers vs Rotary Encoders

The Teensy Audio Tutorial Adapter includes 3 potentiometer knobs that are monitored through the 3 of the Teensy's Analog Input ports, which utilize 10-bit ADCs for a range of 0-1023. It is in the nature of devices of this sort (as well as sensors of various types) when connected directly to be subject to a good deal of noise and "jitter", requiring such techniques as input averaging, filtering, and such to achieve stability.

In addition, there is the issue of matching the ADCs input range to the range of the parameter to be controlled, generally by scaling or mapping one range to another. While this can work acceptably when the input range is much larger than that of the target—for example, mapping 0-1023 to a target range of 0-100 allows for 10 values of the input to be mapped to a single target value, resulting in much greater stability—though not always.

However, when the target range is close to or exceeds the input range, the use of analog inputs becomes problematic (this is to ignore that fact that potentiometers are often non-linear in their response). For example, in the case of a digital delay of maximum time 1 second, if one wishes to set to an exact 1 millisecond value, the task becomes nearly impossible with a 10-bit converter, and even more so for a 2 second digital delay with that granularity.

In such cases, a very good and precise alternative is provided by a Rotary Encoder, which has the appearance of an analog knob, but rather than outputting voltages, it monitors "clicks" as it is turned clockwise or counter-clockwise, without the limit of stop points. In use, it can increment or decrement a numeric value by adding or subtracting the number of clicks from the current value.

So, the encoder can be used to set very precise values of the parameter to which it is assigned, regardless of range. But, with a range of 1000 or more points, this would be quite tedious if the value could only be changed by +/- 1 per click. Fortunately, many such devices also include a pushbutton function for the knob, such that, with successive pushes, the increment value can be moved through a range of values such as (1, 5, 10, 20) or (1, 10, 50, 100, 500), etc. depending on the situation.

The KY-050 encoders used in this project are very common, and obtainable from many online sources, including ProtoSupplies. In addition, there are numerous online guides to how encoders work and how to use them. For this project, a class "Rotary Encoder" was designed, based on the Arduino open-source library:

<https://github.com/luni64/EncoderTool>

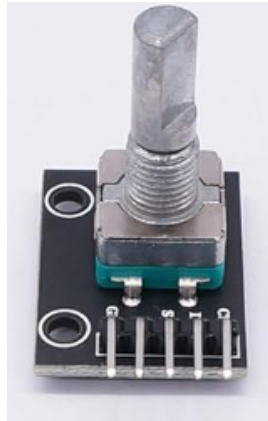
KY-050

The KY-050 rotary encoder is a popular model for TeensyDuino users. The device includes a rotary knob and a push-button switch and can be purchased in both a PCB-mounted unit with on-board pullup resistors (see schematic below), or as a non-PCB "bare" unit. Both formats are available at Amazon from multiple vendors.

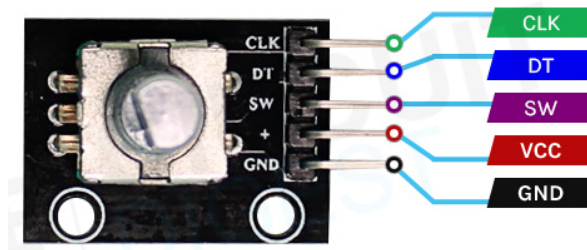
PCB Mounted

https://www.amazon.com/dp/B07F26CT6B?psc=1&ref=ppx_yo2ov_dt_b_product_details

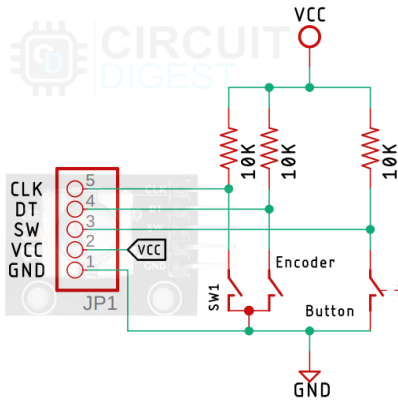
Front Pin View



Front Pin ViewPin Labels



Circuit Schematic



Tested successfully with Teensy 4.1 Proto Board using EncoderTool library (<https://github.com/luni64/EncoderTool>) and following test program, and with both Vcc (3.3v) and Gnd connected:

```
#include "ArduinoSerialMacros.h"
#include "EncoderTool.h"
using namespace EncoderTool;

// Rotary Encoder
#define ENCODER_PIN_CLK          26
#define ENCODER_PIN_DT          25
#define ENCODER_PIN_SWITCH      27
#define ENCODER_INCR_SMALL      ((long) 1)
#define ENCODER_INCR_LARGE      ((long) 10)
#define ROTARY_VAL_MIN          ((long) 0)
#define ROTARY_VAL_MAX          ((long) 100)

PolledEncoder enc;

long encoderIncr = ENCODER_INCR_SMALL;
long rotaryVal = ROTARY_VAL_MIN;
long rotaryValPrev = ROTARY_VAL_MIN;
long encoderPos = 0;
long encoderPosPrev = 0;
long count = 0;

void setup() {

    Serial.begin(115200);
    enc.begin(ENCODER_PIN_CLK, ENCODER_PIN_DT, ENCODER_PIN_SWITCH);
}

void loop() {

    enc.tick();                                     // polled encoder -> call tick() as often as possible

    if (enc.buttonChanged()) {                      // do we have a new button state?
        if (enc.getButton() == LOW) {
            if (encoderIncr == ENCODER_INCR_SMALL) {
```

```

        encoderIncr = ENCODER_INCR_LARGE;
    } else {
        encoderIncr = ENCODER_INCR_SMALL;
    }
    Serial.print("encoderIncr: ");
    Serial.println(encoderIncr);
}

}

if (enc.valueChanged()) {                // do we have a new encoder value?
    encoderPos = enc.getValue();
    if (encoderPos > encoderPosPrev) {
        rotaryVal += encoderIncr;
    } else {
        rotaryVal -= encoderIncr;
    }
    encoderPosPrev = encoderPos;

    if (rotaryVal < ROTARY_VAL_MIN) {
        rotaryVal = ROTARY_VAL_MIN;
        enc.setValue(0);
        encoderPosPrev = 0;
    } else if (rotaryVal > ROTARY_VAL_MAX) {
        rotaryVal = ROTARY_VAL_MAX;
        enc.setValue(0);
        encoderPosPrev = 0;
    }

    if (rotaryVal != rotaryValPrev) {
        rotaryValPrev = rotaryVal;

        Serial.print("rotaryVal: ");
        Serial.println(rotaryVal);
        Serial.println();
    }
}
}
}

```



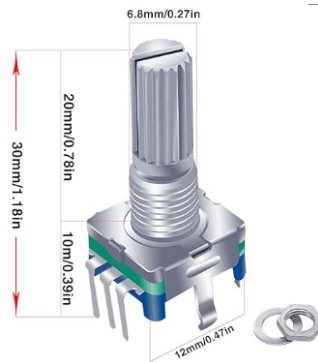
Note that the EncoderTool enables the Teensy pullups on all inputs, such that the ones on the circuit boards are superfluous, and the above code works even when the Teensy's 3.3v signal is disconnected* from the encoder PCB. The library, when in polled mode, does a very good and accurate job of debouncing both the rotary and pushbutton switches. *However, it seems that the encoder works less well when rotated in the counter-clockwise direction. Further testing is in order.

Further tests revealed that, while both the PCB and non-PCB (below) versions work, the PCB version, which includes resistors that are theoretically out of circuit when the + terminal is disconnected, require 2 or 3 clicks (sometimes more) when the knob is turned in order to change the value, whereas the non-PCB version changes value reliably on a single click. The non-PCB version, therefore, would seem to be the best choice.

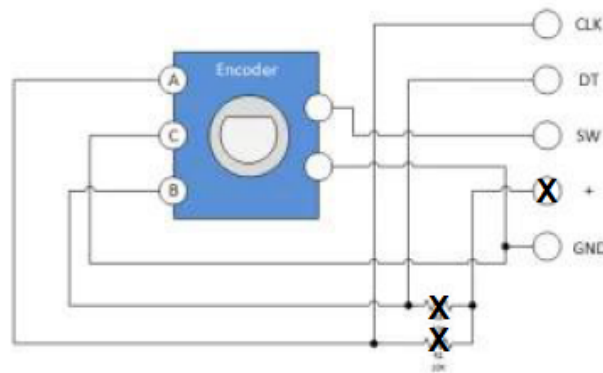
Non-PCB Original Component

https://www.amazon.com/dp/B07F24TRYG?psc=1&ref=ppx_yo2ov_dt_b_product_details

Because of the lack of need for the PCBs pullups, and also the orientation of the header pins, which would require that the PCB be mounted on a target board with the knobs facing outward rather than upward, experimentation with the non-PCB version is indicated (as of 10/19/23). Also, should test with both internal and external pullups. (Done on 11/5/23 and confirms non-PCB version as best choice.)



Testing with these original, non-PCB components reveals that they operate the same as the PCB version, and, as noted before, because of the use of pullup inputs on the Teensy 4.1, it is not necessary to connect to the 3.3V power on the Teensy. The pin arrangement of the non-PCB component is as shown here:



Note that the example program shown in the previous section works equally well with the non-PCB component.

RotaryEncoder Class

AudioPlatform Application

TeensyDuino

Application uses TeensyDuino 1.59.0



Source code for files referenced on GitHub is also included in the AudioPlatform software distribution at <https://github.com/Neurofrantic/AudioPlatform>

TeensyDuino Installation Location

`C:/UserName/AppData/Local/Arduino15/packages/teensy/hardware/avr/1.59.0/libraries`

TeensyDuino 1.59.0 Library Modifications and Additions

libraries/Audio

Audio.h

- Near line 128, add the following include statement:
`#include "play_wav.h"`

effect_delay_ext.cpp / effect_delay_ext.h

- Replace the TeensyDuino 1.59.0 modules with updated code from:
https://github.com/h4yn0nnym0u5e/Audio/blob/feature/AudioEffectDelayExternal/effect_delay_ext.cpp
and
https://github.com/h4yn0nnym0u5e/Audio/blob/feature/AudioEffectDelayExternal/effect_delay_ext.h

extmem.cpp / extmem.h

- Add the following 2 modules to the TeensyDuino 1.59.0 Audio library:
<https://github.com/h4yn0nnym0u5e/Audio/blob/feature/AudioEffectDelayExternal/extmem.cpp>
and
<https://github.com/h4yn0nnym0u5e/Audio/blob/feature/AudioEffectDelayExternal/extmem.h>

play_wav.cpp / play_wav.h

- Add the following 2 modules to the TeensyDuino1.59.0 Audio library:
https://github.com/FrankBoesing/Teensy-WavePlayer/blob/main/play_wav.cpp
and
https://github.com/FrankBoesing/Teensy-WavePlayer/blob/main/play_wav.h
Moved to Libraries/MediaPlayers. User action no longer necessary.

libraries/ILI9341_t3

- Remove ILI9341_t3.cpp and LIL9341_t3.h from
**C:/UserName/AppData/Local/Arduino15/packages/teensy/hardware/avr/1.59.0/
libraries**

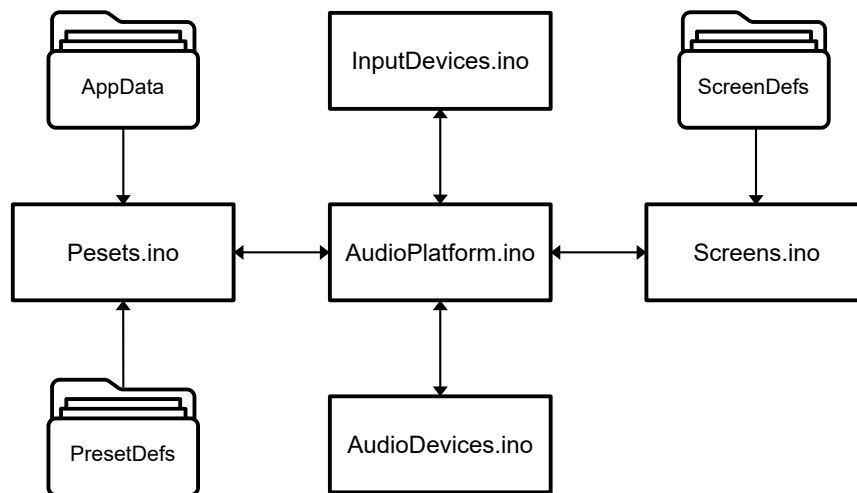


ILI9341_t3 is a library for the 320x240 ILI9341 display which is supplanted in AudioPlatform by the RA8875 800x480 display. Normally, the presence of ILI9341_t3.cpp and ILI9341_t3.h in the TeensyDuino library would not cause any issues. But in the case of AudioPlatform, the Libraries folder of AudioPlatform includes a folder named ILI9341_fonts-master, which includes a number of fonts not present in ILI9341_t3, and the presence of ILI9341_t3 in the TeensyDuino library creates a conflict.

libraries/RA8875

- Modify RA8875/src/settings/RA8875UserSettings.h line64 to remove the comment “//” prefix, thereby enabling the use of the RA8875 capacitive touch screen:
#define USE_FT5206_TOUCH//capacitive touch screen

Core Components

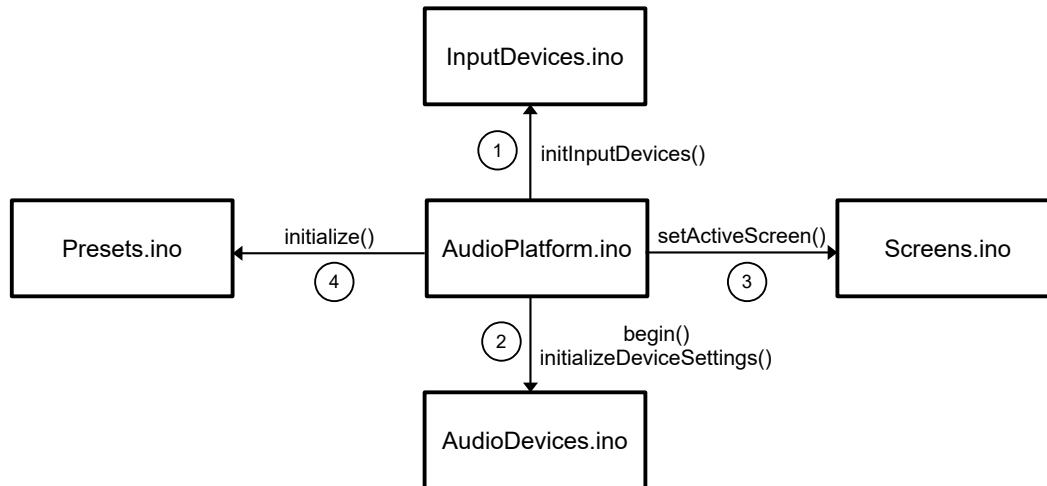


Application Core Components



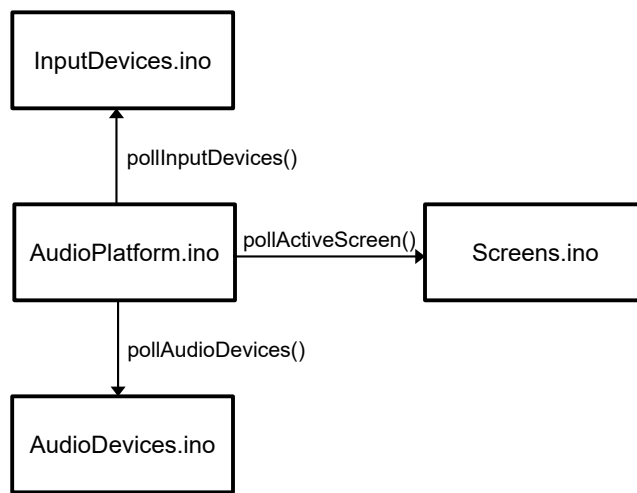
Normally, in mixed C and C++ projects, the statement *extern "C"* is generally required ensure compatibility when linking C++ code with code written in C. In the context of the Arduino ecosystem, where the code is inherently C++-based, using *extern "C"* is not automatically required because the environment implicitly handles C++ compatibility.

Initialization



Runtime Polling

General discussion of polling strategies, top to bottom.

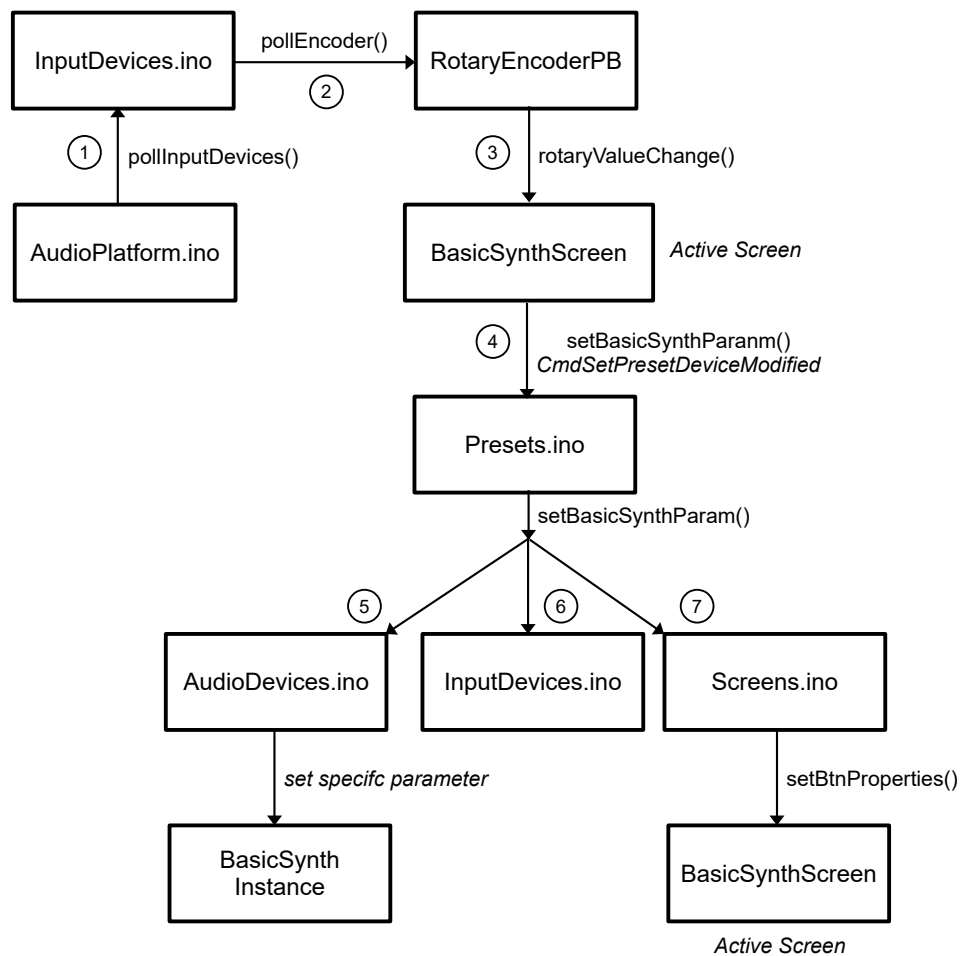


“Popup” Screens

Active Screen→”PopupScreen1”→”PopupScreen2”→...→”TopMostPopupScreen”

Input Device Polling

Rotary Encoders



Real Time Clock

TouchScreen Polling Restrictions

In a given polling sequence, the function *DisplayScreenBase::touched()* must be called only once. In certain instances, an “outer” polling function may call other functions in an arbitrary order only if the called functions themselves do not also call function *DisplayScreenBase::touched()*. There are a number of classes which may be polled from the outer polling function which themselves also call *DisplayScreenBase::touched()* within their polling sequences. For example, the following sequence will fail because *DisplayScreenBase::touched()* is called both in the outer poll and also in function *pollSdBankSelect()*. So, the inner call to *DisplayScreenBase::touched()* in *pollSdBankSelect()* will not return a correct result.

```
if (DisplayScreenBase::touched(WaitTimeAfterKeyTouch)) {
    DisplayScreenBase::getTouchedPoint(pointX, pointY);

    if (sdBankFileSelect.pollEnabled) {
        pollSdBankSelect();
    }
    .
    .
    .
```

In such cases, it is necessary to arrange code such that only one call is made to xxx, such as in the following sequence:

```
if (presetNameEdit.pollEnabled) {
    pollPresetNameEdit();

} else if (bankFileNameEdit.pollEnabled) {
    pollBankFileNameEdit();

} else if (sdBankFileSelect.pollEnabled) {
    pollSdBankSelect();

} else if (DisplayScreenBase::touched(WaitTimeAfterKeyTouch)) {
    DisplayScreenBase::getTouchedPoint(pointX, pointY);
    .
    .
    .
```

Not really specific to Teensy or AudioPlatform, but rather ideas to take note of and possibly apply in development.

Lambda Functions

From <https://stackoverflow.com/questions/4324763/can-we-have-functions-inside-functions-in-c>

In current versions of c++ (C++11, C++14, and C++17), you can have functions inside functions in the form of a lambda:

```
int main() {
    // This declares a lambda, which can be called just like a function
    auto print_message = [](std::string message)
    {
        std::cout << message << "\n";
    };

    // Prints "Hello!" 10 times
    for(int i = 0; i < 10; i++) {
        print_message("Hello!");
    }
}
```

Lambdas can also modify local variables through **capture-by-reference**. With capture-by-reference, the lambda has access to all local variables declared in the lambda's scope. It can modify and change them normally.

```
int main() {
    int i = 0;
    // Captures i by reference; increments it by one
    auto addOne = [&] () {
        i++;
    };

    while(i < 10) {
        addOne(); //Add 1 to i
        std::cout << i << "\n";
    }
}
```

That said, you can have local classes, and they can have functions (non-static or static), so you can get this to some extent, albeit it's a bit of a kludge:

```

int main() // it's int, dammit!
{
    struct X { // struct's as good as class
        static void a()
        {
        }
    };

    X::a();

    return 0;
}

```

However, I'd question the practice. Everyone knows (well, now that you do, anyway :)) C++ doesn't support local functions, so they are used to not having them. They are not used, however, to that kludge. I would spend quite a while on this code to make sure it's really only there to allow local functions. Not good.

2014 Update:

With the rise of C++11 a while back, you can now have local functions whose syntax is a little reminiscent of JavaScript:

```

auto fac = [] (unsigned int val) {
    return val*42;
};

```

See Also:

For various bodies and return types:

<https://www.programiz.com/cpp-programming/lambda-expression#:~:text=C%2B%2B%20Lambda%20Function%20With%20Return,%20return%20a%20%2B%20b%3B%20%7D%3B>

Example: C++ Lambda - Explicit Return Type

```

#include<iostream>
using namespace std;

int main() {

    // lambda function with explicit return type 'double'
    // returns the sum or the average depending on operation
    auto operation = [] (int a, int b, string op) -> double {
        if (op == "sum") {
            return a + b;
        }
        else {

```

```
        return (a + b) / 2.0;
    }
};
```

Appendix

References

AudioPlatform Source and Documentation

[AudioPlatform Source Code](#)

[AudioPlatform Documentation](#)

ProtoSupplies

[ProtoSupplies Web Site](#)

[Prototyping System For Teensy 4.1](#)

[Teensy 4.1 Fully Loaded For Prototyping System](#)

[Project System For Teensy 4.1](#)

[Teensy 4.x Audio Adapter](#)

PJRC / Teensy

[PJRC](#)

[Teensy User Forum](#)

[Teensy 4.1](#)

[TeensyDuino](#)

[TeensyDuino Libraries](#)

[Teensy Audio Shield](#)

[Teensy Audio Tutorial](#)

Teensy DIY Audio Projects and Kits

Teensy Forum

[TeensyMix Synth - DIY 8 Voice Synthesizer](#)

[Shruti II Teensy 4.1 DIY Synthesizer](#)

[Polyhonic Teensy DIY Sampler](#)

[Teensy Super Audio Board](#)

[MicroDexed Touch Synthesizer](#)

[Polyhonic Teensy DIY Sampler](#)

YouTube

[DIY Dirtywave M8 headless using a Teensy 4.1](#)

[DX7 Clone on Teensy 4.0](#)

[Electrotechnique TSynth - Teensy 4.1 based DIY Synthesizer](#)

[Teensy 4.1 Drone Synth](#)

[Teensy Midi Step Sequencer](#)

[FALA Synthesizer - 8 Voice DIY Teensy Synth](#)